

From Ontology to Property Graph

A cookbook for building ontology-grounded knowledge graphs with LLM agents

Compiled for a GraphRAG practitioner · September 2026

Running example, code and outputs: [cookbook/examples/](#)

How to use this cookbook

The sources this book distils

The running example: SupplyKG

Part I · 1. Three graph data models, one set of facts

- 1.1 Directed edge-labelled graphs (RDF)
- 1.2 Heterogeneous graphs
- 1.3 Property graphs
- 1.4 Converting between the models
- 1.5 Graph datasets and named graphs

2. Designing the ontology: a 7-step method

- Step 1 · Determine domain and scope
- Step 2 · Consider reuse
- Step 3 · Enumerate terms
- Step 4 · Define classes and the hierarchy
- Step 5 · Define properties
- Step 6 · Define facets (cardinality, value type, domain/range)
- Step 7 · Create instances
- 2.1 Class, property value, or node?
- 2.2 Naming conventions
- 2.3 Design checklist

3. OWL semantics you must understand before mapping

- 3.1 Classes, hierarchy, disjointness
- 3.2 Property characteristics
- 3.3 Domain and range are inference rules, not constraints
- 3.4 Restrictions: `some`, `only`, cardinality, `value`
- 3.5 Primitive vs defined classes
- 3.6 The open-world assumption (OWA)
- 3.7 No unique-name assumption (no UNA)
- 3.8 Data properties and datatypes
- 3.9 OWL vs LPG: the clash table

4. Schema, identity and context

- 4.1 The three kinds of schema
- 4.2 Identity
- 4.3 Context and qualified facts

Part II · 5. Property-graph modelling judgement

- 5.1 Nodes and labels
- 5.2 Relationships
- 5.3 Refactoring
- 5.4 Eliminating duplicate data: property → node
- 5.5 Checklist before accepting a model

6. The OWL → LPG mapping recipes

- R1 · `owl:Class` → node label
- R2 · `rdfs:subClassOf` → three options
- R3 · Defined classes (`owl:equivalentClass`) → materialised labels
- R4 · `owl:inverseOf` and `owl:SymmetricProperty` → one stored direction
- R5 · `rdfs:subPropertyOf` → specific type (+ optional parent)
- R6 · `owl:TransitiveProperty` → direct edges only, closure at query time
- R7 · n-ary relationships → intermediate node (or edge properties)

R8 · `owl:DatatypeProperty` → typed node property
R9 · Functional properties, cardinality and `someValuesFrom` → validation
R10 · Identity: IRIs, keys, `hasKey`, inverse-functional → uniqueness
R11 · `rdfs:domain` / `rdfs:range` → allowed patterns (extraction schema + checks)
R12 · Enumerations and value partitions → property with allowed values, or reference nodes
R13 · Annotations → schema catalogue and prompt descriptions
R14 · Disjointness → label-conflict checks
Master mapping table

7. Implementing the mapping: owl2lpg, n10s and Neptune

7.1 Compile the ontology once, use it everywhere
7.2 The n10s route (Neo4j)
7.3 The Neptune route
7.4 Which route?

Part III · 8. Ontology-grounded agentic construction

8.1 Design principles from the course
8.2 Where the ontology plugs in
8.3 Structured sources: proposer + critic + deterministic loader
8.4 Unstructured sources: schema-constrained extraction
8.5 The schema guard and repair loop
8.6 Provenance, confidence and claims
8.7 The lexical graph and linking

9. Entity resolution

9.1 The pipeline
9.2 Metrics
9.3 A worked experiment
9.4 The LLM judge (for the ambiguous band)
9.5 ER in Cypher
9.6 Schema mapping for structured sources

10. Validation: OWL vs SHACL vs Cypher checks

10.1 Three validators, one batch
10.2 The same checks on the LPG
10.3 A validation pipeline that works

Part IV · 11. Quality and drift monitoring

11.1 Accuracy
11.2 Coverage
11.3 Coherency
11.4 Drift: emergent schema vs ontology
11.5 A minimal quality dashboard

12. From graph to answers, and lessons from industry

12.1 Competency questions as acceptance tests
12.2 text2cypher with the compiled schema
12.3 Hybrid retrieval over an ontology-typed graph
12.4 Lessons from the Amazon Product Graph

Appendix A · End-to-end checklist

Appendix B · Corrections to the sources

Appendix C · References

Appendix D · Files in `examples/`

How to use this cookbook

This book assumes you already know RAG, GraphRAG and graph retrieval (PPR-style seeding, community summaries, text2cypher). It fills the gap in between: **how a formal ontology becomes a labeled property graph (LPG), and how agents fill that graph without wrecking it.**

It is organised as four parts:

Part	Chapters	You will be able to...
I. Foundations	1–4	Read and write OWL. Know what every axiom <i>means</i> (not just what it looks like). Know the three ways a schema can behave (infer, validate, emerge).
II. The mapping	5–7	Turn any OWL construct into an LPG pattern using recipes R1–R14, generate constraints and validation queries, and target Neo4j or Neptune.
III. Population	8–10	Run an ontology-constrained multi-agent extraction pipeline with a schema guard, repair loop and entity resolution.
IV. Operation	11–12	Measure quality, detect schema drift, and query the result for QA.

Every example is real. The running example, *SupplyKG*, is an OWL ontology in `examples/supplykg.ttl`. Every “Output” block in this book was produced by running the scripts in `examples/` using `rdflib`, the `owlrl` OWL 2 RL reasoner, and `pyshacl`:

```
cd cookbook && python3 -m venv .venv && .venv/bin/pip install rdflib pyshacl owlrl matplotlib
cd examples
../.venv/bin/python owl2lpg.py prompt           # schema text for LLM prompts
../.venv/bin/python owl2lpg.py instances        # MERGE statements
# other modes: instances-reasoned, neptune, constraints, checks, ontology-graph, json
../.venv/bin/python reason_demo.py               # what OWL infers
../.venv/bin/python validate_demo.py              # OWL vs SHACL on a bad LLM batch
../.venv/bin/python schema_guard.py               # validating LLM extraction JSON
../.venv/bin/python er_demo.py out.png            # entity-resolution metrics
```

Callout boxes used throughout:

RECIPE A step-by-step transformation you can apply directly.

TIP A practice that saves time or prevents a class of bugs.

PITFALL Something that looks right and silently isn't.

SOURCE NOTE Where an idea comes from, or where a source is wrong or out of date.

 RDF resource / OWL class  LPG node  n-ary / event node  literal / condition  agent / process  lexical / document  error / violation

Diagram legend used throughout the cookbook.

The sources this book distils

Each concept is taught from exactly one source, and the rest of the book points back to it.

Source	Used for
Noy & McGuinness, <i>Ontology Development 101</i>	Method, competency questions, class-vs-property decisions (Ch. 2)
DeBellis, <i>Protégé Pizza Tutorial v3.2</i>	OWL semantics: restrictions, characteristics, defined classes, open world (Ch. 3)
Hogan et al., <i>Knowledge Graphs</i> §2.1, §3.1–3.3, §7.1–7.3	Data models, schema kinds, identity, context/reification, quality (Ch. 1, 4, 11)
Neo4j GraphAcademy, <i>Graph Data Modeling Fundamentals</i> (modules 2, 3, 5, 6)	LPG modelling judgement (Ch. 5)
Neo4j Labs <i>neosemantics (n10s)</i> docs + source	Reference OWL→LPG mapping (Ch. 6–7)
AWS Neptune user guide	Neptune constraints on the target LPG (Ch. 7)
DeepLearning.AI × Neo4j, <i>Agentic Knowledge Graph Construction</i> (public notebooks)	Multi-agent construction design (Ch. 8)
Stanford CS520: Kejriwal (entity resolution), Dong (Amazon Product Graph) + AutoKnow paper	ER metrics and industrial lessons (Ch. 9, 12)

The running example: SupplyKG

A mid-sized manufacturer wants a knowledge graph over its **suppliers, products, components, contracts, people and customer reviews**, so an agent can answer supply-chain questions. The data comes from ERP exports (CSV), contract PDFs and review text. It is small enough to read in full, and it still contains every hard case: a class hierarchy, a defined class, inverse, symmetric, transitive and functional properties, two n-ary relationships, and a lexical layer.

Competency questions (CQs). These are the questions the graph must answer. We come back to them in every chapter.

#	Competency question	What it forces into the model
CQ1	Which suppliers provide components used (at any depth) in RoboArm 200?	transitive <code>hasComponent</code> , supplier ↔ component link
CQ2	What did Globex pay per unit for Bolt X-7, and since when?	qualified (n-ary) supply agreement
CQ3	If Germany has a port strike, which assembled products are at risk?	supplier location, bill-of-materials traversal
CQ4	Who was Globex's CTO in 2021?	temporally qualified employment
CQ5	Which products have reviews rated ≤ 2 that mention a component?	lexical graph → domain graph links
CQ6	Which companies compete with Globex?	symmetric relation

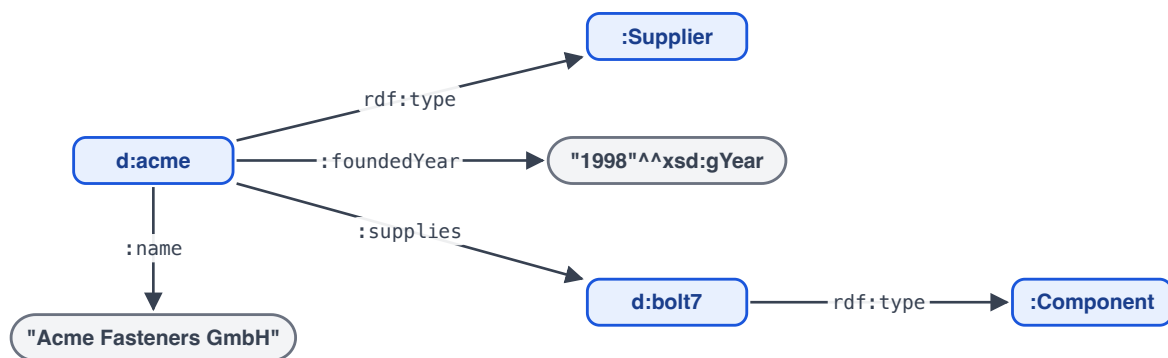
Part I · 1. Three graph data models, one set of facts

Before mapping anything, be precise about what each model *can* say. Hogan et al. (§2.1) describe three models. Every mapping decision later in the book is a trade between them.

1.1 Directed edge-labelled graphs (RDF)

A **directed edge-labelled graph** (DELG) is a set of triples (*subject*, *predicate*, *object*). RDF is the standard DELG. Its nodes are IRIs (global identifiers), literals (typed values) or blank nodes (anonymous).

```
d:acme a :Supplier ;  
      :name "Acme Fasteners GmbH" ;  
      :foundedYear "1998"^^xsd:gYear ;  
      :supplies d:bolt7 .  
d:bolt7 a :Component .
```



Directed edge-labelled graph (RDF). Everything is a triple, including the type of a node and its literal attributes. There is nowhere to hang `since 2021` on the `:supplies` edge.

Three properties of this model matter for mapping:

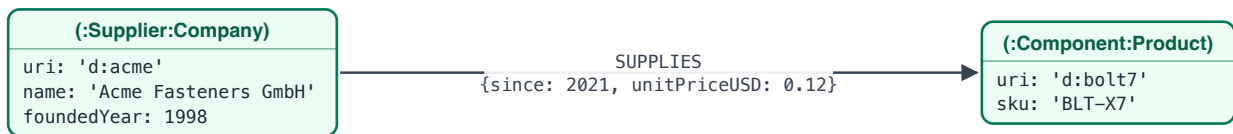
1. **Edges have no identity.** The edge set is a *set* of triples, so `(acme, supplies, bolt7)` can exist once. You cannot state it twice, and you cannot attach `since 2021` to it without extra machinery (§4.3).
2. **Types are just edges.** `rdf:type` is an ordinary edge, so a node can have zero, one or many types.
3. **Everything is a node or a literal.** Any value that should be traversable (a country, a currency) is a node with an IRI. A literal is a dead end.

1.2 Heterogeneous graphs

A heterogeneous graph gives each node **exactly one** type, as part of the model, and labels edges. Many GNN libraries and knowledge-graph embedding toolkits (PyG `HeteroData`, DGL) assume this. Since Acme is both a `Supplier` and a `Company`, a heterogeneous graph forces you to pick one. **Plan a projection step** if you will train GNNs on your LPG.

1.3 Property graphs

Formally, a property graph is $G = (V, E, L, P, U, e, l, p)$. Nodes and **edges have their own IDs**, both carry a **set of labels**, and both carry **key/value properties**.



The same facts as a labeled property graph. Types become labels, literals become properties, and the relationship itself can carry key/value pairs.

```
CREATE (a:Supplier:Company {uri:'d:acme', name:'Acme Fasteners GmbH', foundedYear:1998})
-[:SUPPLIES {since:2021, unitPriceUSD:0.12}]->
(b:Component:Product {uri:'d:bolt7', sku:'BLT-X7'})
```

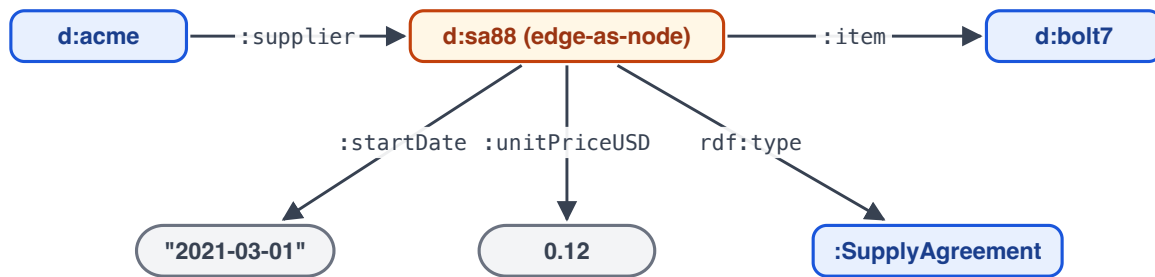
What you gain: edge identity (two parallel `SUPPLIES` edges for two contracts), qualifiers on edges, and compact queries. What you lose, shown in Hogan's own example (Fig. 2.3): **an edge property value is a literal, never a node**. If you write `company: 'LATAM'` on a flight edge, the airline stops being an entity you can traverse, deduplicate or link to Wikidata.

RULE OF THUMB Anything that has, or could ever need, an IRI becomes a **node**. Only true scalars (numbers, dates, codes, free text) become properties.

1.4 Converting between the models

The conversion is mechanical, which is what makes an ontology usable as an LPG schema.

LPG construct	RDF / DELG encoding
node ID	IRI
node label	<code>rdf:type</code> edge to a class
node property <code>k = v</code>	edge <code>node -k-> literal</code>
edge with properties	a new node for the edge, with one edge to each endpoint and one per property
edge property	edge from the edge-node to the value



Converting an LPG edge that has properties into RDF: the edge becomes a node with one edge per endpoint and one per property. Going the other way, "relation nodes" with exactly one from/to pair can be collapsed back into edges.

The reverse direction (RDF → LPG) finds **relation nodes**, collapses them into edges, turns literal-valued edges into properties, and turns `rdf:type` edges into labels. `owl2lpg.py` (Ch. 7) does exactly this, but it lets you decide which classes are relation nodes, instead of guessing.

KEEP THE REVERSE MAPPING Document the LPG → RDF direction as well. With it you can export the LPG to Turtle and run an OWL reasoner and SHACL over it (Ch. 10). Those are the tools that catch the bugs agents introduce.

1.5 Graph datasets and named graphs

An RDF *dataset* is a default graph plus named graphs (*name*, *graph*). The name is an IRI that can itself carry metadata:

```

:batch_2026_09_24 { d:acme :supplies d:bolt7 . }           # facts from one LLM run
{ :batch_2026_09_24 :extractedBy :claudie ; :source <file:contracts/C-88.pdf> . }
  
```

Named graphs are the most flexible way to record **provenance for a whole batch**. The LPG equivalent is an `(:Extraction)` node, or an `extraction_id` property on every element a run created (§4.3, Ch. 8).

2. Designing the ontology: a 7-step method

Distilled from Noy & McGuinness, “Ontology Development 101”, applied to SupplyKG.

An ontology has **classes** (concepts), **properties** (called slots in the paper), and **facets** (restrictions on properties). An ontology plus its instances is a knowledge base. Three ground rules frame everything that follows:

1. There is no single correct model. The right one depends on the application.
2. Development is iterative.
3. Concepts should track real things. Nouns in your domain sentences tend to become classes, and verbs tend to become relations.

FRAMES VS OWL Ontology 101 was written for *frame* systems (Protégé-2000). There, properties belong to classes, domain means “attached to”, and facets behave like constraints. That is closer to how an LPG behaves than OWL is. Use Ontology 101 for **design discipline**, and Chapter 3 for **what the OWL axioms actually mean**.

Step 1 · Determine domain and scope

Answer four questions: *What domain? What for? Which questions must it answer? Who uses and maintains it?* For SupplyKG: supply-chain risk and procurement QA, used by an agent and by analysts, maintained by the data team. **Out of scope:** HR payroll, accounting, logistics routing.

Scope determines detail. The paper’s wine example shows it: an NLP use needs synonyms, a restaurant needs retail price, a cellar manager needs wholesale price. For SupplyKG, the LLM extraction use case means we **must** carry synonyms (`skos:altLabel "vendor"`) and definitions (`rdfs:comment`) into the ontology, because the extractor’s prompt is built from them (Ch. 8).

Competency questions set scope and later act as a litmus test (see the CQ table in the front matter). The most specific things that appear as *answers* to CQs are good candidates for **individuals**.

CQS AS TESTS Write each CQ as an openCypher query *before* you populate the graph. Those queries become your acceptance tests (Ch. 11) and your few-shot examples for text2cypher.

Step 2 · Consider reuse

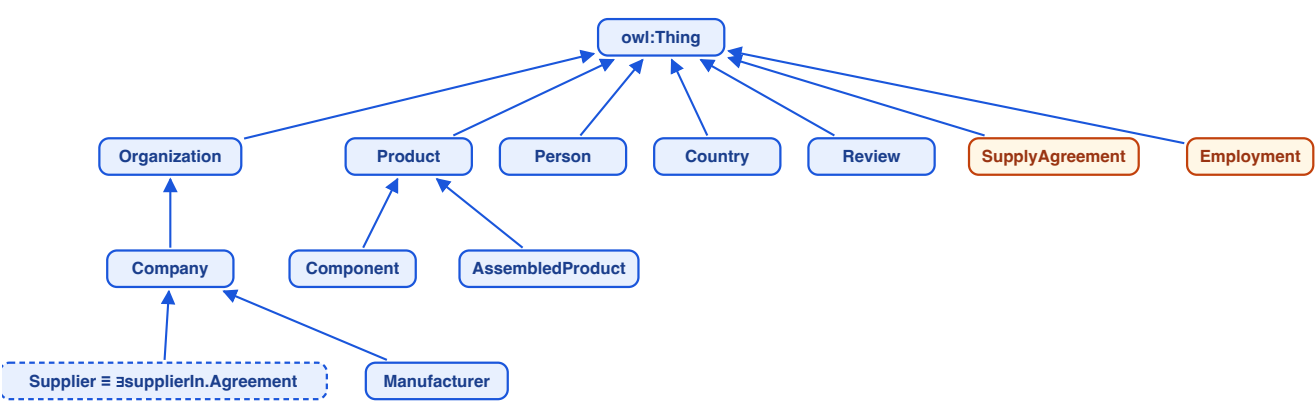
Look for existing vocabularies: schema.org (`Organization` , `Product`), GoodRelations, the W3C Organization ontology, PROV-O for provenance, OWL-Time for intervals, and SKOS for synonym tables. Reuse matters most when other systems already commit to a vocabulary. SupplyKG defines its own terms for readability and uses SKOS for synonyms.

Step 3 · Enumerate terms

Brainstorm without judging: *supplier, vendor, component, part, SKU, bill of materials, contract, unit price, review, rating, country, headquarters, CTO, employment, competitor, manufacturer*. Don't decide yet what is a class and what is a property.

Step 4 · Define classes and the hierarchy

Pick the terms that name things which **exist on their own**. Arrange them by asking: “If something is an *X*, is it necessarily a *Y*?”



SupplyKG class hierarchy (arrows = `rdfs:subClassOf`). Orange = reified n-ary relationships. Dashed = defined class, whose membership is inferred, never asserted.

You can work top-down (Organization → Company → Supplier), bottom-up (Component, AssembledProduct → Product), or **middle-out**, starting from the most salient concepts. Middle-out is usually easiest, because middle-level concepts are the most descriptive.

Hierarchy checks from the paper, applied to SupplyKG:

Check	SupplyKG decision
Siblings at the same level of generality	Supplier / Manufacturer are siblings; Company / Supplier are not
2–12 direct subclasses per class	Company has 2 ✓. A class with exactly one subclass is a smell.
Synonyms are not classes	“vendor”, “provider” are <code>skos:altLabel</code> on Supplier, not classes
No singular/plural mix-ups	Component, never Components
No cycles (A ⊆ B ⊆ A means A = B)	—
Declare disjointness	Organization, Product, Person, ... are pairwise disjoint; Component ⊥ AssembledProduct
Classes shouldn't depend on transient state	no DelayedSupplier class; delay is a property of a shipment

SOURCE ERRATUM Ontology 101 §4.1 states the subclass definition backwards (“...every instance of B is also an instance of A”). The correct reading, stated elsewhere in the paper: if $A \sqsubseteq B$, every instance of **A** is an instance of **B**.

Step 5 · Define properties

Most remaining terms become properties. Four kinds:

Kind	SupplyKG example
intrinsic	<code>sku</code> , <code>rating</code>
extrinsic	<code>name</code> , <code>foundedYear</code>
parts	<code>hasComponent</code>
relationships to other individuals	<code>manufactures</code> , <code>locatedIn</code> , <code>competitorOf</code>

Attach each property to the **most general class whose instances all have it**: `foundedYear` on `Organization` , `sku` on `Product` . Not higher (`sku` on `owl:Thing`), and not lower (on each product subclass).

Step 6 · Define facets (cardinality, value type, domain/range)

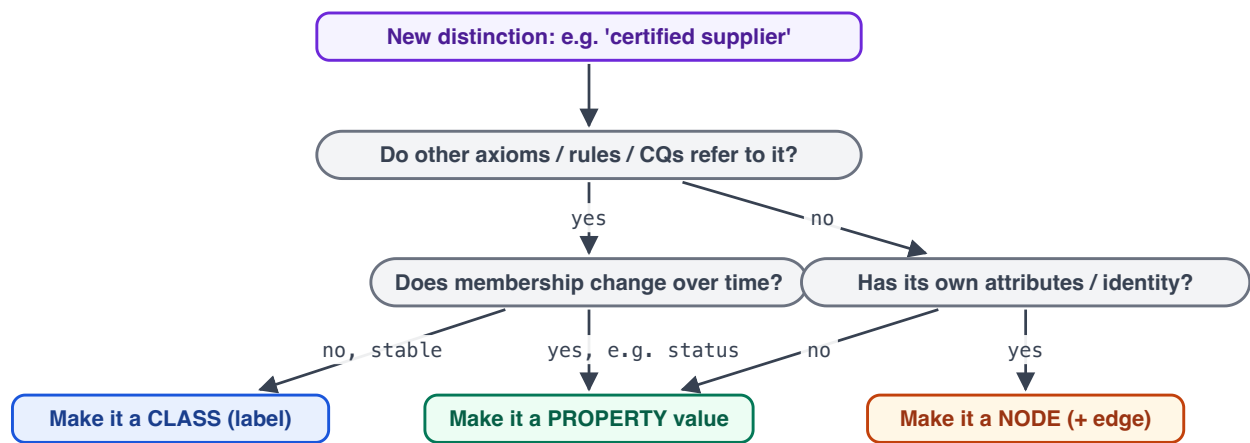
- **Cardinality.** `headquarteredIn` is single-valued (functional). `locatedIn` is multi-valued.
- **Value type.** String, number, boolean, enumerated, or instance of a class (an object property).
- **Domain/range.** Use the most general classes that fit, but no more general than that. Drop a subclass if its parent is already listed. If *all* subclasses of A are listed, replace them with A.
- **Inverse properties.** `manufactures` / `manufacturedBy` . Declaring both helps data entry, but in an LPG you store only one direction (recipe R4).
- **Defaults.** OWL has none. It is monotonic. Defaults belong in the ingest pipeline or as LPG property defaults.

Step 7 · Create instances

Pick a class, create the individual, fill its properties (see `examples/instances.ttl`).

2.1 Class, property value, or node?

This is the decision people get wrong most often. It combines Ontology 101 §4.5–4.6 with the property → node refactoring rule from Neo4j GraphAcademy (Ch. 5):



Decision flow: class, property value, or node? Adapted from Noy & McGuinness §4.5 (classes shouldn't depend on transient state) and the Neo4j "property → node" refactoring rule.

- **Make it a class** if the distinction is referenced by other axioms, rules or CQs (“reviews of *assembled* products”), and an instance’s membership rarely changes.
- **Make it a property value** if it describes state that changes (`status: 'on-hold'`), or is a scalar.
- **Make it a node** if the value is shared by many instances, is filtered or traversed on, or has its own attributes (`Country` , `Currency`).
- **Individual vs class**. If the concepts form a natural hierarchy (regions within countries), they are **nodes linked by PART_OF** , not a class hierarchy, because instances have no “sub-instance” relation.

DROPPING THE QUALIFIER `Bavaria ⊆ Germany` is wrong: Bavaria is not a *kind of* Germany. Either write `BavarianRegion ⊆ GermanRegion` , or better, use `(:Region {name: 'Bavaria'})-[:PART_OF]->(:Country {name: 'Germany'})` .

2.2 Naming conventions

Element	OWL convention	LPG convention
Class	UpperCamelCase , singular	label UpperCamelCase
Object property	lowerCamelCase verb phrase (<code>hasComponent</code>)	relationship UPPER_SNAKE_CASE (<code>HAS_COMPONENT</code>)
Datatype property	lowerCamelCase	property lowerCamelCase
Inverse pair	<code>hasX / isXOf</code>	one type only
Siblings	all or none repeat the parent’s name (<code>RedWine / WhiteWine</code>)	same

Avoid abbreviations, and don’t put “class” or “property” inside names.

2.3 Design checklist

- ☐ Every CQ is answerable by a path through the schema.

- ☐ Every class has a definition (`rdfs:comment`) and its synonyms (`skos:altLabel`). The extractor needs them.
- ☐ Sibling classes are disjoint wherever the domain says so.
- ☐ No class encodes transient state.
- ☐ Every object property has a domain and a range, set as specific as is true.
- ☐ Functional properties are marked. Keys (`sku` , `isoCode`) are marked functional, and ideally `owl:hasKey` .
- ☐ Facts that carry qualifiers (price, date, role, source) are modelled as n-ary classes (§4.3).

3. OWL semantics you must understand before mapping

Distilled from the Protégé Pizza Tutorial v3.2 (DeBellis), with Hogan et al. §3.1.1 and §3.2.2.

OWL is Description Logic, a decidable fragment of first-order logic. The mental model that avoids most mistakes is set theory: **a class is a set, a property is a binary relation, an individual is an element.**

`owl:Thing` is the universal set and `owl:Nothing` is the empty set. Properties are first-class entities that exist independently of classes. That is the key difference from object-oriented attributes, and from how labels work in an LPG.

The ontology is the **T-Box** (terminology). The instance data is the **A-Box** (assertions).

3.1 Classes, hierarchy, disjointness

```
:Company rdfs:subClassOf :Organization .      # every Company is necessarily an Organization
:Supplier rdfs:subClassOf :Company .
[] a owl:AllDisjointClasses ; owl:members ( :Organization :Product :Person :Country :Review ) .
```

- A hierarchy is **not a tree**. Multiple inheritance is normal, and an individual can belong to many classes.
- **Classes overlap unless declared disjoint**. Without the axiom above, nothing prevents a node from being both a `Company` and a `Product`.
- Disjointness is **inherited**: `Supplier` is also disjoint with `Product`.

PROBE CLASSES To test a model, create a deliberately impossible class, such as `Probe` $\sqsubseteq$ `Component` $\sqcap$ `AssembledProduct`. If the reasoner doesn't mark it unsatisfiable ($\equiv$ `owl:Nothing`), your disjointness axioms don't say what you think they say.

3.2 Property characteristics

Characteristic	Meaning	SupplyKG example	What the reasoner does	LPG consequence
inverse	$a P b \Rightarrow b Q a$	manufactures / manufacturedBy	infers the other direction	store one direction (R4)
functional	at most one value per subject	headquarteredIn	two values $\Rightarrow$ infers the values are the same	cardinality check (R9)
inverse functional	at most one subject per value	(keys, e.g. sku)	two subjects $\Rightarrow$ infers sameAs	uniqueness constraint + ER (R10)
transitive	$a P b, b P c \Rightarrow a P c$	hasComponent	infers the closure	store direct edges, query *1.. (R6)
symmetric	$a P b \Rightarrow b P a$	competitorOf	infers the reverse	store once, query undirected (R4)
asymmetric	$a P b \Rightarrow \neg b P a$	—	inconsistency on violation	validation query
reflexive / irreflexive	every / no individual relates to itself	—	Δ reflexive forces its domain $\equiv$ owl:Thing	rarely useful
sub-property	$a P b \Rightarrow a Q b$	headquarteredIn $\sqsubseteq$ locatedIn	infers the parent edge	materialise or query both (R5)

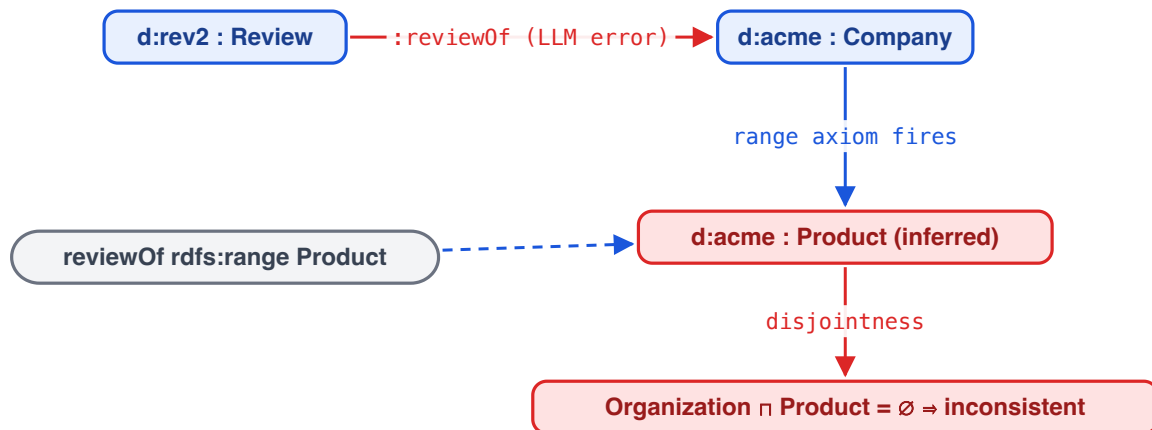
PRECISION The Pizza tutorial says a transitive property “cannot be functional”. Strictly, OWL 2 DL *forbids* cardinality or functional axioms on non-simple (e.g. transitive) properties to keep reasoning decidable. It is a syntactic restriction, not a logical impossibility.

3.3 Domain and range are inference rules, not constraints

This is the most important sentence in the chapter. `rdfs:domain` and `rdfs:range` do **not** reject bad data. They **classify** the things they touch:

```
:reviewOf rdfs:domain :Review ; rdfs:range :Product .
d:rev2 :reviewOf d:acme .          # an extractor error: a review "of" a company
```

The reasoner does not complain. It concludes that `d:acme` is a `Product`. You only get an error because we also declared `Organization` and `Product` disjoint:



Domain/range are inference rules, not checks. A wrong extraction silently retypes the target; only a disjointness axiom turns it into a visible error. Reproduced by `examples/reason_demo.py`.

Output from `examples/reason_demo.py`:

```
# Domain misuse: an extractor writes d:rev2 :reviewOf d:acme
acme inferred to be a :Product -> True
OWL-RL inconsistency messages: 1
- Disjoint classes https://example.org/supplykg#Organization and https://example.org/supplykg#Product
  have a common individual https://example
```

MULTIPLE DOMAINS MEAN INTERSECTION Two `rdfs:domain` axioms on the same property mean the subject must belong to **both** classes. For “either”, write one domain with a union (`Pizza or IceCream`). A naive mapping to an LPG “allowed source labels” list flips the meaning.

3.4 Restrictions: `some`, `only`, `cardinality`, `value`

Restrictions are anonymous classes: “the set of things that have property P related to class C”.

```
Class: AssembledProduct   SubClassOf: hasComponent some Component      # ∃ at least one
Class: VeganPizza         EquivalentTo: Pizza and (hasTopping only VeganTopping) # ∀ all fillers
Class: SingleSourced       EquivalentTo: Component and (suppliedBy exactly 1 Supplier)
Class: GermanSupplier     EquivalentTo: Supplier and (headquarteredIn value d:DE)
```

- `some` does not forbid other fillers.
- `only` is **vacuously true** when there are no fillers. A pizza with no toppings is a `VeganPizza`.
- **All restrictions on a class are ANDed.** Two separate `only` restrictions intersect. Write one `only (A or B)` instead.
- Cardinality counts only fillers that are **known to be different** individuals (see §3.7).

3.5 Primitive vs defined classes

	Syntax	Meaning	Reasoner behaviour
Primitive	SubClassOf	necessary conditions: <i>if</i> x is a C, then ...	nothing is classified <i>into</i> C
Defined	EquivalentTo	necessary and sufficient : anything that satisfies ... is a C	individuals and classes are classified into C

In SupplyKG, `Supplier` is **defined**:

```
:Supplier owl:equivalentClass [ a owl:Restriction ;  
  owl:onProperty :partyToAgreementAsSupplier ; owl:someValuesFrom :SupplyAgreement ] .
```

The instance data only says `d:acme a :Company` and `d:acme :partyToAgreementAsSupplier d:sa88`. The reasoner adds the rest.

Output from `examples/reason_demo.py`: inferred facts about instances (OWL 2 RL).

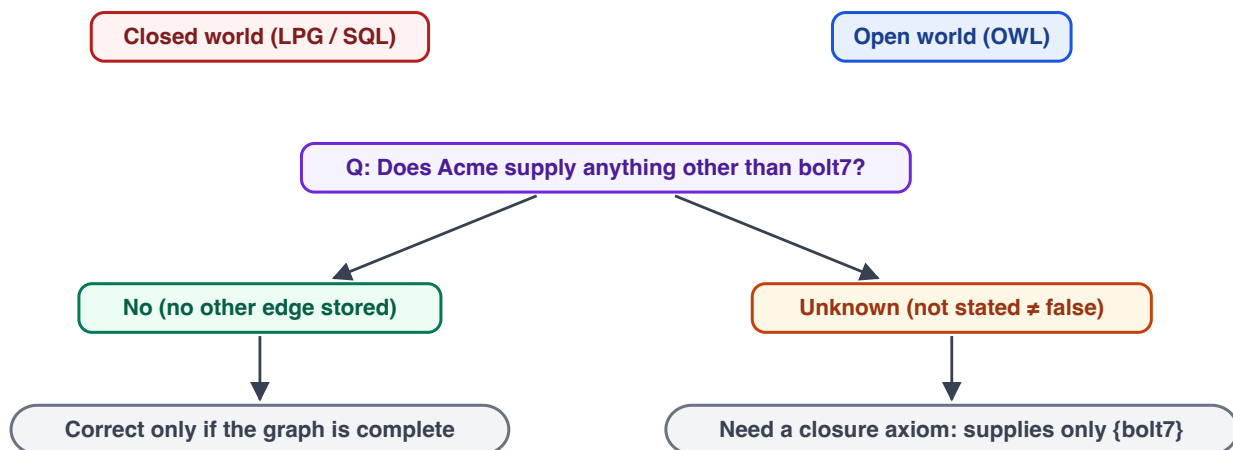
```
d:acme      :locatedIn      d:DE          # from headquarteredIn ⊆ locatedIn  
d:acme      rdf:type        :Organization  # from Company ⊆ Organization  
d:acme      rdf:type        :Supplier      # from the DEFINED class  
d:arm200    :hasComponent   d:bolt7        # transitivity: arm200 → gear5 → bolt7  
d:arm200    :manufacturedBy d:globex       # inverse  
d:arm200    rdf:type        :Product  
d:bolt7     rdf:type        :Product  
d:gear5     rdf:type        :Product  
d:globex    :competitorOf   d:initech     # symmetry  
d:globex    rdf:type        :Company  
d:globex    rdf:type        :Organization  
d:initech   rdf:type        :Organization
```

NORMALISATION PATTERN Keep *asserted* classes primitive, with one parent each, and let *defined* classes create the polyhierarchy by inference. In an LPG pipeline, run the reasoner once before loading and materialise the inferred labels (recipe R3).

3.6 The open-world assumption (OWA)

Under the OWA, **not stated** $\neq$ **false**. OWL was designed for the Web, where more data may exist elsewhere. The Pizza tutorial's classic demo: `MargheritaPizza` has `hasTopping some Mozzarella` and `some Tomato`, yet it is **not** classified as `VegetarianPizza`. The reasoner assumes it might have other, unstated toppings. The fix is a **closure axiom**:

```
Class: MargheritaPizza SubClassOf:  
  hasTopping some MozzarellaTopping, hasTopping some TomatoTopping,  
  hasTopping only (MozzarellaTopping or TomatoTopping) # closure
```



The same question under both world assumptions. LLM-populated graphs are incomplete by construction, so a closed-world "no" is really "not extracted".

Databases, SQL and openCypher are **closed-world**: `NOT EXISTS { (a)-[:SUPPLIES]->() }` means "no". Hogan (§3.1.1) describes a middle ground, the **local closed-world assumption**: declare *specific* parts of the graph complete ("our contract list is complete"), and treat everything else as open.

LLM-POPULATED GRAPHS ARE OPEN-WORLD WHETHER YOU LIKE IT OR NOT If an extractor missed a contract, a closed-world query answers "Acme supplies nothing else" with full confidence. Record which parts of the graph are complete (a `complete: true` flag per source, or completeness statements). Make the QA agent say "not found in the graph" instead of "no" (Ch. 11).

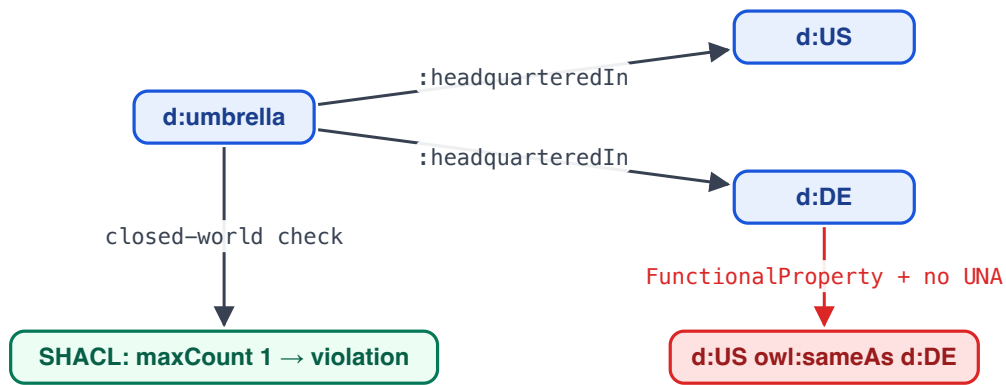
3.7 No unique-name assumption (no UNA)

OWL does not assume that different IRIs denote different things. That has two consequences which surprise everyone coming from databases:

1. **Functional properties merge, they don't reject.** Two HQ values mean the two countries are the *same*.
2. **Cardinality can't count** unless individuals are declared `owl:differentFrom` / `owl:AllDifferent`.

Real output from `examples/validate_demo.py`, when an LLM says Umbrella Corp is headquartered in both the US and Germany:

```
## OWL 2 RL view of the bad batch
sameAs merges inferred: [('DE', 'US'), ('Umbrella Corp', 'Umbrella Corporation'), ('d:DE', 'd:US')]
```



Functional properties merge, they don't reject. OWL concludes that the US and Germany are the same country; SHACL reports the defect instead.

The reasoner concluded that **Germany and the United States are the same country**, and propagated that into the literals as well. That is correct OWL semantics, and it is almost never what a KG engineer wants. It motivates the core rule of Part II:

CORE RULE Use OWL axioms for **meaning and inference** (hierarchy, inverse, transitivity, defined classes). Use **closed-world validation** (SHACL, or Cypher checks on the LPG) for **data integrity** (cardinality, required properties, allowed endpoints, value ranges). Never rely on OWL alone to catch an extraction error.

3.8 Data properties and datatypes

- A data property relates an individual to a literal: `xsd:string`, `xsd:integer`, `xsd:decimal`, `xsd:date`, `xsd:dateTime`, `xsd:boolean`.
- Data properties can be functional, but not inverse, transitive, and so on.
- Prefer `xsd:integer` / `xsd:decimal` over `xsd:int` / `xsd:float`.
- Facets express ranges: `hasRating some xsd:integer[>= 1, <= 5]`.
- **Wrong literal datatypes** are among the most common errors. A string in an integer-typed property makes the ontology inconsistent. The same goes for LLM output ("12 cents"), which the schema guard in Ch. 8 catches.

3.9 OWL vs LPG: the clash table

#	OWL	Typical LPG	What to do
1	Open world	Closed world	Record completeness. Phrase QA answers accordingly.
2	Domain/range infer types	Endpoints unchecked, or rejected by a constraint	Treat as validation (R11)
3	Several domains = intersection	Allowed-labels lists are a union	Normalise to a single (union) class first
4	No UNA: sameAs inferred	Node identity is physical	Entity resolution before MERGE . Functional violation = error.
5	Membership can be inferred (defined classes)	Labels are asserted	Reasoner pre-pass, materialise labels (R3)
6	Subclass transitivity	MATCH (:Company) misses :Supplier -only nodes	Materialise ancestor labels, or query the hierarchy (R2)
7	Inverse is a separate named relation	Every edge is traversable both ways	Store one direction (R4)
8	Transitive closure inferred	No closure	*1. . paths, or materialised closure with refresh (R6)
9	Monotonic, no defaults	Mutable, defaults easy	Keep provenance so extracted facts can be retracted
10	One contradiction $\Rightarrow$ whole KB inconsistent	Bad data is local	Validate batches, reason over clean exports
11	Annotations on classes	Labels carry no metadata	Keep a schema catalogue or (:Class) meta-nodes (R13)
12	“label” = human-readable name (rdfs:label)	“label” = node type	Say <i>node label</i> vs <i>rdfs:label</i> explicitly

4. Schema, identity and context

Distilled from Hogan et al. §3.1–3.3.

Graphs let you postpone a schema. When you add one later, it can do three quite different jobs:

Kind	Job	Standard	In this book
Semantic schema	define meaning, license inference	RDFS / OWL	Ch. 3; recipes R2–R6
Validating schema	define constraints, find violations	SHACL / ShEx	Ch. 10; recipes R9, R11, R14
Emergent schema	extract the latent structure from the data	quotient graphs, bisimulation	§4.1.3; drift monitoring in Ch. 11

A mature KG uses all three: the ontology says what things mean, shapes say what a batch must look like before it is accepted, and emergent summaries show what the graph has actually become.

4.1 The three kinds of schema

4.1.1 Semantic schema: the four RDFS rules

Axiom	Rule it licenses	SupplyKG
<code>c rdfs:subClassOf d</code>	$x \text{ type } c \Rightarrow x \text{ type } d$	<code>Supplier</code> $\sqsubseteq$ <code>Company</code>
<code>p rdfs:subPropertyOf q</code>	$x p y \Rightarrow x q y$	<code>headquarteredIn</code> $\sqsubseteq$ <code>locatedIn</code>
<code>p rdfs:domain c</code>	$x p y \Rightarrow x \text{ type } c$	<code>reviewOf</code> $\rightarrow$ <code>Review</code>
<code>p rdfs:range c</code>	$x p y \Rightarrow y \text{ type } c$	<code>reviewOf</code> $\rightarrow$ <code>Product</code>

These are **inference rules** (Ch. 3.3). OWL adds equivalence, inverse, disjointness, characteristics, keys and property chains on top.

4.1.2 Validating schema: shapes

A **shape** picks **target** nodes (all instances of a class, subjects of a property, results of a query) and places **constraints** on them: cardinality, datatype, value range, and references to other shapes. A node conforms if it satisfies every constraint of its shape. Shapes can be **open** (other properties allowed) or **closed** (unknown properties are violations).

```
:SupplyAgreementShape a sh:NodeShape ; sh:targetClass :SupplyAgreement ;
  sh:property [ sh:path :agreementBuyer ; sh:class :Company ; sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path :agreementItem ; sh:class :Component ; sh:minCount 1 ; sh:maxCount 1 ] ;
  sh:property [ sh:path [ sh:inversePath :partyToAgreementAsSupplier ] ; sh:minCount 1 ;
    sh:message "Agreement has no supplier" ] ;
  sh:property [ sh:path :unitPriceUSD ; sh:datatype xsd:decimal ; sh:minExclusive 0 ; sh:maxCount 1 ] .
```

CLOSED SHAPES CATCH INVENTED PROPERTIES An LLM that adds `color: "grey"` to a product violates a **closed** shape, and passes an open one. Use closed shapes for extraction batches and open shapes for the curated core.

VALIDATING AFTER INFERENCE HIDES ERRORS If you run RDFS inference before validating, `rdfs:range` retypes wrong targets so that they pass `sh:class` checks. Chapter 10 shows this happening with a real run: two defects vanish.

Recursion and negation. Shapes can reference shapes (an Event is valid only if its Venue is valid). Mixed with negation, this can be paradoxical. Hogan’s “barber” example: a node conforms if it shaves someone who is not a barber, and Bob shaves only himself. Keep extraction shapes non-recursive.

4.1.3 Emergent schema: what the graph actually looks like

A **quotient graph** partitions nodes (for example by their label set) and keeps an edge between partitions if *some* member pair is connected. It always *simulates* the data: every real edge has a summary edge. But it can over-generalise. A **bisimulation** adds the reverse condition (every member has every summary edge), at the cost of more partitions.

The label-set quotient is one openCypher query. It works on Neo4j and Neptune:

```
MATCH (a)-[r]->(b)
RETURN labels(a) AS fromLabels, type(r) AS rel, labels(b) AS toLabels, count(*) AS n
ORDER BY n DESC
```

Compare the result with the schema generated from the ontology (Ch. 7). **Any row that is not a legal pattern is drift:** an invented relationship, a wrong endpoint, or a missing label. Chapter 11 turns this into a monitoring job.

4.2 Identity

4.2.1 Persistent identifiers

When two sources both say `Santiago`, you need global identifiers. Hogan’s recommendations:

- Use **IRIs** for entities, in a namespace you control (`https://example.org/supplykg/data/acme`), or reuse persistent IDs (DUNS, LEI, GTIN/EAN for products, ISO 3166 for countries, Wikidata QIDs).
- Keep **entity IRIs** separate from **document IRIs**. The contract PDF is not the supplier. For persistence, use a redirect layer (PURL-style) so the identifier stays fixed when servers move.

4.2.2 External identity links and the UNA

`owl:sameAs` links two IRIs for the same entity. OWL makes `sameAs` symmetric and transitive, so you can link every source to one **hub** (such as Wikidata) instead of to each other. Because OWL has **no unique-name assumption**, `FunctionalProperty`, `InverseFunctionalProperty` and `owl:hasKey` *infer* `sameAs` rather than raising errors (Ch. 3.7). Use `owl:differentFrom` to block merges you know are wrong.

In an LPG you choose one of three representations:

Option	How	Pros	Cons
(a) merge	one physical node	simplest queries	loses per-source provenance; hard to undo
(b) link	(<code>:Mention</code>)-[:REFERS_TO]->(<code>:Entity</code>) or (<code>x</code>)-[:SAME_AS]->(<code>y</code>)	reversible, auditable	queries must follow the link
(c) cluster id	<code>canonicalId</code> property on each node	cheap, reversible	every query must group by it

DEFAULT FOR AGENT-BUILT GRAPHS Use (b) between the **lexical layer** and the **domain layer** (`(:Mention)-[:REFERS_TO {score}]->(:Supplier)`), and (a) inside the domain layer only when confidence is high and a human or a deterministic key (SKU, LEI) confirms it. See Ch. 9.

4.3 Context and qualified facts

Context is the scope in which a fact is true: **time** (2019–2022), **place**, or **provenance** (according to contract C-88). A **qualifier that belongs to the fact itself** (the *role* “CTO”, the *price* 0.12) is different from context. Both need somewhere to live.

Running fact: *Alice worked at Globex as CTO from 2019-01-15 to 2022-06-30, according to contract.pdf.*

Direct representation (event node): the default

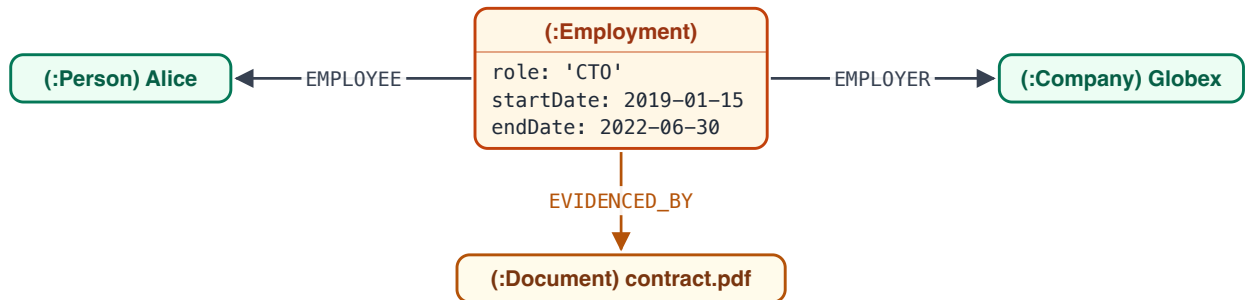
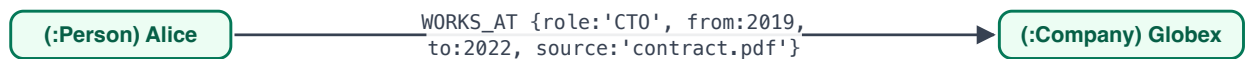
Turn the relationship into a real domain entity, here an `Employment`. This is what SupplyKG does for `Employment` and `SupplyAgreement`.

```
d:empl a :Employment ; :employee d:alice ; :employer d:globex ; :role "CTO" ;
  :startDate "2019-01-15"^^xsd:date ; :endDate "2022-06-30"^^xsd:date ;
  prov:wasDerivedFrom <file:contract.pdf> .
```

```
(:Person {name:'Alice Chen'})<-[:EMPLOYEE]-(:Employment {role:'CTO', startDate:date('2019-01-15'),
  endDate:date('2022-06-30')})-[:EMPLOYER]->(:Company {name:'Globex Robotics'})
```

Edge properties (LPG only)

```
(:Person)-[:WORKS_AT {role:'CTO', from:date('2019-01-15'), to:date('2022-06-30'),
  source:'contract.pdf'}]->(:Company)
```



Top: qualifiers as relationship properties (compact, but the source is a string and nothing can point at the edge). Bottom: an intermediate event node (one hop longer, but provenance, multiple stints and retraction are first-class).

RDF-only styles (for reading other people's ontologies)

```

# Standard reification: the statement is a node; the base triple is NOT asserted by this
:st1 a rdf:Statement ; rdf:subject :alice ; rdf:predicate :worksAt ; rdf:object :globex ; :role "CTO" .
# n-ary via rdf:value
:alice :worksAt :w1 . :w1 rdf:value :globex ; :role "CTO" .
# Singleton property
:alice :worksAt_1 :globex . :worksAt_1 :singletonPropertyOf :worksAt ; :role "CTO" .
# Named graph: context for many triples at once
:g1 { :alice :worksAt :globex . } { :g1 :validFrom "2019"^^xsd:gYear ; prov:wasDerivedFrom :S . }
# RDF-star: the quoted triple is a node, but it has no edge identity
<< :alice :worksAt :globex >> :role "CTO" ; :validFrom "2019"^^xsd:gYear .
  
```



Two RDF-only reification styles, for comparison. Left: standard reification (4+ triples per fact, base edge must be asserted separately). Right: singleton property (one predicate per fact, which bloats the schema). Both collapse to an LPG edge with properties or an event node.

THE “TWO STINTS” TEST Hogan’s example: Bachelet was president 2006–2010 **and** 2014–2018. With RDF-star (and with MERGE on $(s, TYPE, o)$ in Cypher) there is a single quoted triple or edge, so the four dates cannot be paired. Event nodes, parallel LPG edges and named graphs all pass this test. RDF 1.2 “reifiers” ($\sim :stint1 \{ | \dots | \}$) also fix it. Check the current W3C drafts before relying on that syntax.

Choosing a representation

Style	What the extra node denotes	Base edge kept?	Multiple stints?	Can provenance be a node?	Hops
Event node	a domain entity (Employment)	replaced	✓	✓	+1
Edge properties	—	✓	✓ (parallel edges; don't MERGE on endpoints)	✗ (string only)	0
RDF reification	the statement	must be asserted separately	✓	✓	+1
Singleton property	the edge (predicate)	✓	✓	✓	0, but schema bloat
Named graph	a set of triples	✓	✓	✓	0
RDF-star	the triple (no identity)	✓	✗	✓	0

DECISION RULE (RECIPE R7) Use an **event node** if any of these holds: (1) something must *point at* the fact (evidence, retraction, a review of the contract); (2) there are ≥ 2 qualifiers that belong to the fact itself; (3) a participant is an entity, not a literal (a currency node, a document node); (4) the fact is a CQ answer in its own right (“list all supply agreements”). Otherwise use **edge properties**.

Asserted vs reported. A reified statement isn't necessarily asserted. You can reify a claim precisely to say it is *disputed*. For LLM extraction, that suggests a useful pattern: write low-confidence facts as `(:Claim)` nodes (not edges) and promote them to domain edges after validation or review (Ch. 8.6).

Part II · 5. Property-graph modelling judgement

Distilled from Neo4j GraphAcademy, Graph Data Modeling Fundamentals, modules 2, 3, 5 and 6.

An ontology says what is *true*. An LPG model also has to make the CQs **fast and simple to query**.

GraphAcademy's method is driven by use cases. List the questions, derive the model from them, and re-derive it when a new question arrives. That is exactly what competency questions give you.

5.1 Nodes and labels

- Labels come from the **nouns** in the use cases. Relationship types come from the **verbs**.
- Node properties do three jobs: **identify** the node (anchor a `MATCH`), **filter** during traversal, and **return** data. Counting relationships is not a job for a property.
- Every noun a runtime step touches becomes a property (“retrieve a product by **SKU**, return its **name**”).
- A label is an **anchor**. It narrows the starting set before any traversal. `MATCH (n:Supplier)` scans suppliers only, where `MATCH (n)` scans everything.
- **Labels cannot be parameterised in Cypher.** `country` as a property keeps `WHERE n.country = $c` flexible, while `:DE` as a label does not.

LABEL BUDGET GraphAcademy recommends at most **4 labels per node**. Neo4j stores labels differently beyond that, and access slows down. It also lists **class hierarchies as stacked labels** as an anti-pattern. That tension with OWL is resolved in recipe R2.

5.2 Relationships

- Name relationship types in `UPPER_SNAKE_CASE`, usually as verbs (`MANUFACTURES`, `REVIEW_OF`). Don't use names that could be mistaken for an entity (`RATING` is a noun; `RATED` is better).
- Each relationship type needs a **start label, a type, and an end label**. That triple is exactly the (*domain, property, range*) of an OWL object property.
- **Direction:** every stored relationship has one. Pick the direction that reads naturally. Symmetric relationships are stored once and queried undirected (`-[:COMPETITOR_OF]-`).
- **Relationship properties:** add them only when a use case asks *how* two nodes are related, not merely *whether* they are (`ACTED_IN {role}`, `RATED {rating}`).
- **Fanout:** splitting an entity into a small network of nodes (city, last name) helps some queries. Overdoing it creates **supernodes** with hundreds of thousands of relationships. Ontology-derived “value” nodes (`Country`, `Currency`, `Language`) are exactly the ones that become supernodes.

5.3 Refactoring

You refactor when (1) a use case can't be answered, (2) a new use case arrives, or (3) queries are slow at scale. The steps are: design the new model, write Cypher to transform the graph, then **retest every use case**. Use `PROFILE` and compare the rows at each step. On small graphs, db-hits and timings are distorted by caching.

Add a label from participation, the GraphAcademy example adapted to SupplyKG:

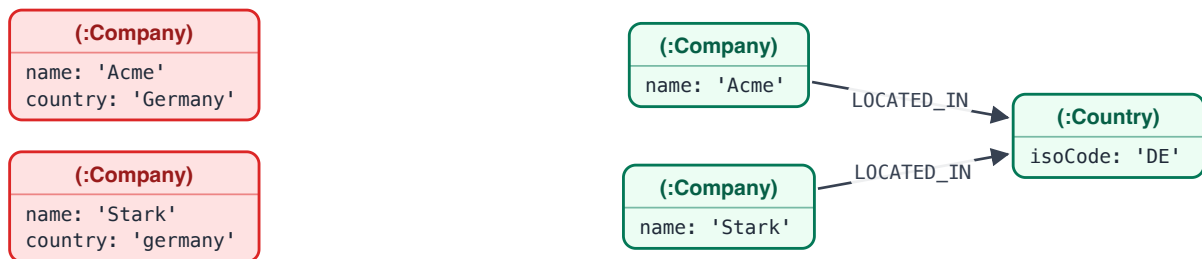
```
// every company that is party to at least one agreement as supplier gets :Supplier
MATCH (c:Company) WHERE EXISTS { (c)-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]->() }
SET c:Supplier
```

This is the LPG equivalent of materialising the OWL **defined** class `Supplier` (recipe R3). The same query works as a nightly job if you don't run a reasoner.

SYNTAX DRIFT GraphAcademy's lessons write `WHERE exists((p)-[:ACTED_IN]->())`. That form is deprecated in Neo4j 5. Use `EXISTS { ... }` or a plain pattern predicate.

5.4 Eliminating duplicate data: property → node

A repeated string property invites spelling drift, and filtering on it forces a scan of every node. The fix is to model the value as a shared node:



Property → node refactoring. Left: a repeated string property invites spelling drift and can't be traversed. Right: one shared node that entity resolution, filters and PageRank-style walks can use.

```
// GraphAcademy's Language/Genre recipe, applied to countries
MATCH (c:Company) WHERE c.country IS NOT NULL
MERGE (k:Country {isoCode: toUpper(c.country)})
MERGE (c)-[:LOCATED_IN]->(k)
REMOVE c.country
```

For list properties: `UNWIND c.certifications AS cert MERGE (x:Certification {name: cert}) MERGE (c)-[:HAS_CERTIFICATION]->(x) SET c.certifications = null`.

When a property should become a node (GraphAcademy's criteria):

Make it a node when...	Keep it a property when...
the value is shared by many nodes (low cardinality)	it is unique per node
a use case filters or anchors on it	it is only returned
you want to traverse through it ("other products from the same country")	it would create an unhelpful supernode
the value is structured (an address)	it is a scalar

IT MATTERS TWICE FOR GRAPHRAG Shared value nodes are also what makes **PPR-style retrieval** work. A random walk can only move from one supplier to another *through* a node, never through a string property.

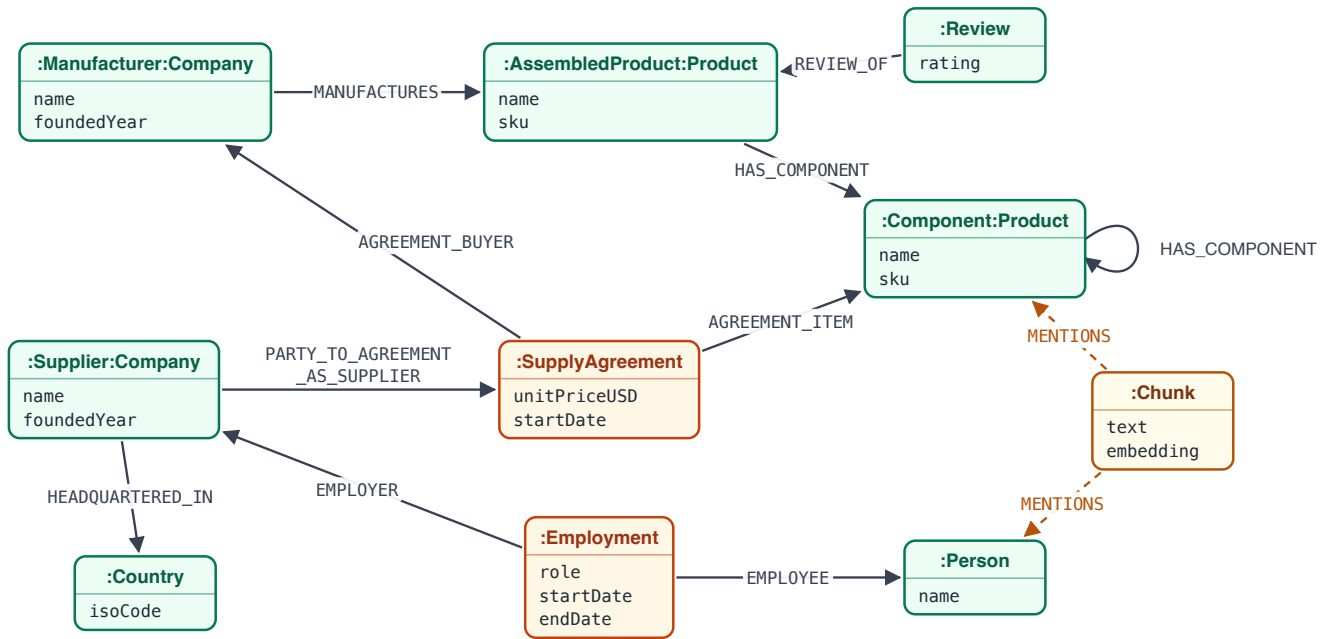
5.5 Checklist before accepting a model

- ☐ Every CQ runs as a single `MATCH` (at most a variable-length path), and is anchored on a label plus a key property.
- ☐ No node has more than 4 labels.
- ☐ No two relationship types are inverses or synonyms of each other.
- ☐ Repeated low-cardinality values are nodes, and unique scalars are properties.
- ☐ Potential supernodes are known. Queries through them are anchored on the *other* end.

6. The OWL → LPG mapping recipes

This chapter is the core of the book. Each recipe takes one OWL construct and gives the LPG pattern, the Cypher, and the pitfall. Everything here is implemented in `examples/owl2lpg.py` (Ch. 7), so you can check each recipe against real output.

Here is the target schema for SupplyKG that these recipes produce:



Target LPG schema for SupplyKG generated by `examples/owl2lpg.py` (simplified). Orange nodes are n-ary relationships promoted to nodes; the yellow `:Chunk` belongs to the lexical graph.

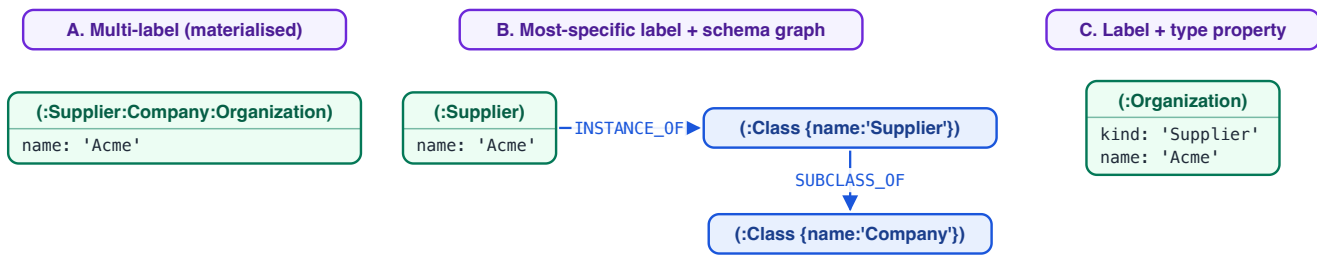
R1 • `owl:Class` → node label

```
:Component a owl:Class ; rdfs:subClassOf :Product ; skos:altLabel "part" .
```

```
(:Component:Product {uri: 'https://example.org/supplykg/data/bolt7', name: 'Bolt X-7', sku: 'BLT-X7'})
```

- Label = the class's local name, `UpperCamelCase`. Keep the full IRI in a `uri` property (R10).
- Put **one base label on every node** (`:Entity`; `n10s` uses `:Resource`). Key lookups can then hit a single index even when the caller doesn't know the type.

R2 • `rdfs:subClassOf` → three options



Three ways to carry `rdfs:subClassOf` into an LPG. A is the default for query speed; B keeps the ontology queryable (n10s style); C suits deep, volatile taxonomies. See the comparison table in recipe R2.

	A. Multi-label (materialised)	B. Leaf label + class graph	C. One label + kind property
Query “all Companies”	<code>MATCH (c:Company)</code>	<code>MATCH (c)-[:INSTANCE_OF]->(:Class)-[:SC0*0..]->(:Class {name:'Company'})</code> or <code>n10s.inference.nodesLabelled</code>	<code>MATCH (o:Organization) WHERE o.kind IN \$companyKinds</code>
Speed	fastest (label scan)	slower (hierarchy walk)	fast, but a filter
Ontology change	relabel instances	edit class nodes only	update the list of kinds
Label budget (≤ 4)	breaks on deep hierarchies	fine	fine
Works on Neptune	✓	✓ (plain Cypher, no n10s)	✓
Best for	shallow, stable top-level classes	deep or volatile taxonomies (product categories, diseases)	many sibling subtypes under one parent

R2 RULE Hybrid. Use option A for the ontology’s *top-level* classes, where $\text{depth} \leq 3$ and a node never needs more than 4 labels (SupplyKG: `Supplier:Company:Organization`). Use option B for any *taxonomy* that is deep, large or edited often. For those, instances link to a category node with `IN_CATEGORY`, and categories link to each other with `SC0`. This satisfies both OWL (the hierarchy is explicit and queryable) and GraphAcademy’s warning against stacked labels.

```
// Option B query on Neptune or Neo4j: every product in category 'Fastener' or any sub-category
MATCH (p:Product)-[:IN_CATEGORY]->(:Category)-[:SC0*0..]->(:Category {name: 'Fastener'})
RETURN p.name
```

R3 • Defined classes (`owl:equivalentClass`) → materialised labels

Membership in a defined class is *inferred*, never asserted, so an LPG will never contain it unless you compute it. There are two ways:

1. **Reasoner pre-pass.** Run OWL 2 RL over the RDF view before loading (`owl2lpg.py instances-reasoned`). The diff against the plain run is exactly the inferred label:

```
< MERGE (n:Entity {uri: ".../acme"}) SET n:Company:Organization, n += {...};
> MERGE (n:Entity {uri: ".../acme"}) SET n:Company:Organization:Supplier, n += {...};
```

2. A Cypher rule per defined class, run after each batch (Ch. 5.3): `MATCH (c:Company) WHERE EXISTS {(c)-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]->()} SET c:Supplier`.

STALE INFERRED LABELS If the agreement is deleted, `:Supplier` stays. Recompute defined labels from scratch per batch (remove the label, then set it again), or record them in an `inferredLabels` list property so you know which labels to recompute.

R4 • `owl:inverseOf` and `owl:SymmetricProperty` → one stored direction

Every LPG edge can be traversed both ways, so storing both `MANUFACTURES` and `MANUFACTURED_BY` doubles storage and creates update anomalies.

- **Inverse pair:** keep the direction that has `rdfs:domain / range` declared (`manufactures`). Normalise incoming `manufacturedBy` facts by swapping them (`owl2lpg.py` does this automatically). Record the inverse name in the schema so `text2cypher` can map “manufactured by” onto `<-[:MANUFACTURES]-`.
- **Symmetric:** store once in a canonical direction (lower IRI → higher IRI) and query undirected:

```
// CQ6
MATCH (:Company {name: 'Globex Robotics'})-[:COMPETITOR_OF]-(rival) RETURN rival.name
```

R5 • `rdfs:subPropertyOf` → specific type (+ optional parent)

`headquarteredIn` $\sqsubseteq$ `locatedIn`. An LPG relationship has exactly one type, so choose:

Option	Write	Query “located in”
materialise parent (default in <code>owl2lpg.py</code>)	<code>HEADQUARTERED_IN + LOCATED_IN</code> <code>{inferred:true}</code>	<code>-[:LOCATED_IN]-></code>
type union at query time	<code>HEADQUARTERED_IN</code> only	<code>-[:HEADQUARTERED_IN LOCATED_IN]-></code> (either type)
<code>n10s</code>	<code>HEADQUARTERED_IN</code> + ontology loaded	<code>n10s.inference.getRels(n,</code> <code>'LOCATED_IN')</code> (Neo4j only)

Materialising the parent keeps `text2cypher` simple. The `inferred: true` flag lets you delete and recompute the materialised edges.

R6 • `owl:TransitiveProperty` → direct edges only, closure at query time

```
// CQ1: suppliers of any component at any depth in RoboArm 200
MATCH (:AssembledProduct {name: 'RoboArm 200'})-[:HAS_COMPONENT*1..]->(c:Component)
    <-[:AGREEMENT_ITEM]-(SupplyAgreement)<-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]-(s:Company)
RETURN DISTINCT s.name, c.name
```

- **Never store the closure.** The reasoner run in R3 infers `arm200 hasComponent bolt7`, and `owl2lpg.py` drops it deliberately, because stored closure edges go stale the moment the BOM changes.
- Bound the depth on big graphs (`*1..8`). If you truly need precomputed reachability, keep it in a separate relationship type (`HAS_COMPONENT_TRANSITIVE`) that is rebuilt from scratch.

R7 · n-ary relationships → intermediate node (or edge properties)

The decision rule is in Chapter 4.3. SupplyKG models `SupplyAgreement` and `Employment` as classes, so they become nodes:

```
// CQ2: what did Globex pay per unit for Bolt X-7, and since when?
MATCH (:Company {name: 'Globex Robotics'})<-[AGREEMENT_BUYER]-(sa:SupplyAgreement)
    -[AGREEMENT_ITEM]->(:Component {name: 'Bolt X-7'})
RETURN sa.unitPriceUSD, sa.startDate

// CQ4: who was Globex's CT0 in 2021?
MATCH (p:Person)<-[EMPLOYEE]-(e:Employment {role: 'CT0'})
    -[EMPLOYER]->(:Company {name: 'Globex Robotics'})
WHERE e.startDate <= date('2021-12-31') AND (e.endDate IS NULL OR e.endDate >= date('2021-01-01'))
RETURN p.name
```

SHORTCUT EDGES If a CQ often only needs “does Acme supply Globex?”, also write a derived `(:Company)-[SUPPLIES_TO {inferred:true}]->(:Company)` edge from the event nodes. It keeps text2cypher queries short. Recompute it; never edit it by hand.

R8 · `owl:DatatypeProperty` → typed node property

XSD	openCypher (Neo4j)	Neptune bulk-load CSV	Note
<code>xsd:string</code>	STRING	String	language tags are lost (see below)
<code>xsd:integer</code> , <code>xsd:int</code>	INTEGER	Int / Long	
<code>xsd:decimal</code> , <code>xsd:double</code>	FLOAT	Double	decimals lose exactness; store money as integer cents if it matters
<code>xsd:boolean</code>	BOOLEAN	Bool	
<code>xsd:date</code>	DATE (date('2021-03-01'))	DateTime	check your engine's temporal support
<code>xsd:dateTime</code>	DATETIME	DateTime	
<code>xsd:gYear</code>	INTEGER	Int	or DATE with a <code>datePrecision</code> flag

- **Multi-valued.** OWL properties are multi-valued unless they are functional. Neo4j allows a homogeneous list. **Neptune openCypher has no list properties**, so non-functional properties there become value nodes (R12) or delimited strings.
- **Language tags.** `rdfs:label "Schraube"@de` has no LPG equivalent. Use `name_de / name_en`, a map, or `(:Label {lang, value})` nodes. n10s can keep tags as `"value@de"` strings (`keepLangTag`).
- **Custom datatypes** (`"12"^^:EUR`) become two properties (`amount`, `currency`) or a `Currency` node.

R9 · Functional properties, cardinality and `someValuesFrom` → validation

In OWL these *infer* things (Ch. 3.7). In the LPG they must *check* things:

OWL axiom	LPG enforcement
<code>FunctionalProperty</code> (object)	query: at most one outgoing edge of that type per node
<code>FunctionalProperty</code> (data)	single scalar, not a list
<code>minCardinality 1 / someValuesFrom</code> (e.g. <code>AssembledProduct ⊆ ∃hasComponent</code>)	query: nodes of that label with no such edge; or a Neo4j existence constraint for properties
<code>exactly n</code>	both queries

```
// generated by owl2lpg.py checks: headquarteredIn is functional
MATCH (x)-[r:HEADQUARTERED_IN]->() WITH x, count(r) AS n WHERE n > 1
RETURN 'HEADQUARTERED_IN functional' AS check, x.uri AS node, n;
// AssembledProduct ⊆ hasComponent some Component
MATCH (p:AssembledProduct) WHERE NOT EXISTS { (p)-[:HAS_COMPONENT]->(:Component) } RETURN p.uri;
```

R10 · Identity: IRIs, keys, `hasKey`, inverse-functional → uniqueness

- Every node carries its IRI in `uri`. On Neo4j that means `CREATE CONSTRAINT entity_uri FOR (n:Entity) REQUIRE n.uri IS UNIQUE`. On Neptune, only `~id` is unique, so use the IRI as `~id` (the `owl2lpg.py neptune` output does this).
- Natural keys (`sku`, `isoCode`, `LEI`): add per-label uniqueness on Neo4j. On Neptune, check uniqueness in the loader, or encode the key into `~id`.
- Mint IRIs deterministically for extracted entities, e.g. `.../data/company/{slug(name)}-{hash(source)}`. Then re-running an extraction is idempotent. After entity resolution (Ch. 9), point the mention at the canonical IRI.
- Blank nodes: skolemise them into IRIs before import. n10s and Neptune Analytics both give blank nodes per-load IDs, so re-importing duplicates them.

R11 · `rdfs:domain / rdfs:range` → allowed patterns (extraction schema + checks)

Turn each object property into a **pattern** (*DomainLabel*)-[*TYPE*]->(*RangeLabel*). The patterns go to three places:

1. The **extraction prompt**, as the only relationships the LLM may emit (Ch. 8).
2. The **schema guard**, which rejects any other pattern before write (Ch. 8.5).
3. **Post-load checks**:

```
MATCH (x)-[r:REVIEW_OF]->(y) WHERE NOT (x:Review AND y:Product)
RETURN 'REVIEW_OF' AS check, x.uri AS from, y.uri AS to;
```

PATTERN CHECKS NEED MATERIALISED ANCESTOR LABELS `WHERE y:Product` only works if components carry `:Product` too (option A in R2). With option B, check through the class graph instead.

R12 · Enumerations and value partitions → property with allowed values, or reference nodes

`owl:oneOf` classes (`Spiciness ≡ {Hot, Medium, Mild}`) and “value partitions” (`Rating ∈ 1..5`):

- **Property + allowed values** when the values are plain codes with no relations: `status IN ['active', 'suspended']`. Check with a query, or a Neo4j property-type constraint.
- **Reference nodes** when values have order or relations (`Hot -[:SPICIER_THAN]-> Medium`), are shared and filtered on, or are needed for PPR walks (Ch. 5.4). This is the GraphAcademy property → node refactor.

R13 · Annotations → schema catalogue and prompt descriptions

`rdfs:label`, `rdfs:comment`, `skos:altLabel` and `skos:definition` on classes and properties carry no instance data, but they are **the most valuable input to the extraction agent**. The DeepLearning.AI course notebooks note that their NER agent confused the `Assembly` label (a physical sub-assembly) with the act of assembling furniture. Their suggested fix is exactly this: a description per label.

- Keep them in the generated schema JSON and prompt text (`owl2lpg.py prompt prints (:Supplier ...)` `[aka: vendor, provider]`).
- Optionally load the T-Box as `(:Class {name, comment}) nodes (owl2lpg.py ontology-graph, n10s-compatible shape)`, so agents can query the schema itself.

R14 · Disjointness → label-conflict checks

```
// generated by owl2lpg.py checks, one per disjoint pair
MATCH (n:Organization:Product) RETURN 'disjoint Organization/Product' AS check, n.uri AS node;
```

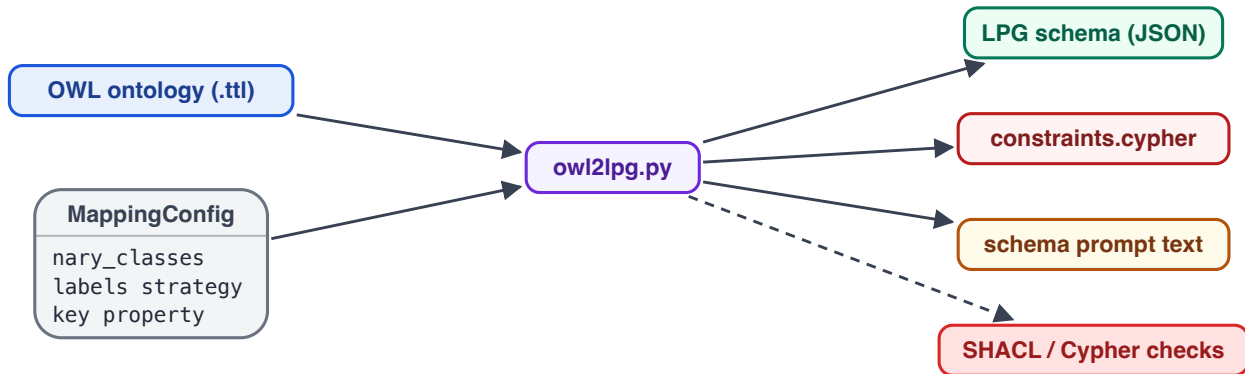
Disjointness is the **only** OWL axiom that turns a silent domain/range misclassification into a visible error (Ch. 3.3). Keep it as a check even if you never run a reasoner.

Master mapping table

OWL / RDFS	LPG	Recipe	Enforced by
<code>owl:Class</code>	label	R1	—
<code>rdfs:subClassOf</code>	ancestor labels (shallow) / <code>SCO</code> class graph (deep)	R2	label materialisation
<code>owl:equivalentClass</code> (defined)	materialised label	R3	reasoner pre-pass or Cypher rule
<code>owl:ObjectProperty</code>	relationship type	R11	pattern list
<code>owl:inverseOf</code>	one direction	R4	normalisation on write
<code>owl:SymmetricProperty</code>	one canonical edge, undirected queries	R4	normalisation on write
<code>rdfs:subPropertyOf</code>	specific type (+ parent { <code>inferred</code> })	R5	materialisation
<code>owl:TransitiveProperty</code>	direct edges only, <code>*1..</code> queries	R6	never store closure
n-ary class / reification	intermediate node, or edge properties	R7	decision rule (4.3)
<code>owl:DatatypeProperty</code>	typed property	R8	type map
<code>owl:FunctionalProperty</code> / cardinality	validation query	R9	checks
IRI / <code>owl:hasKey</code> / inverse functional	<code>uri</code> unique / <code>~id</code>	R10	constraint
<code>rdfs:domain</code> / <code>rdfs:range</code>	allowed patterns	R11	prompt + guard + checks
<code>owl:oneOf</code> / value partition	allowed values or reference nodes	R12	checks
<code>rdfs:label</code> , <code>rdfs:comment</code> , <code>skos:*</code>	schema catalogue, prompt text	R13	—
<code>owl:disjointWith</code> / <code>AllDisjointClasses</code>	label-conflict query	R14	checks
<code>owl:sameAs</code>	<code>REFERS_TO</code> / merge / <code>canonicalId</code>	4.2.2, Ch. 9	ER pipeline

7. Implementing the mapping: owl2lpg, n10s and Neptune

7.1 Compile the ontology once, use it everywhere



The ontology is compiled once into four artefacts: a machine-readable schema, database constraints, a prompt fragment for extraction/text2cypher agents, and validation checks.

The ontology is compiled into four artefacts. Agents never read Turtle. They read the **schema prompt**, and their output is checked against the **schema JSON** and the **checks**.

`examples/owl2lpg.py` (about 360 lines, needing only `rdflib`, plus `owlrl` for the reasoned mode) makes every mapping decision an explicit, overridable setting:

```
@dataclass
class MappingConfig:
    nary_classes: set                # R7: classes that become intermediate nodes
    materialise_superclass_labels: bool = True # R2 option A
    materialise_superproperties: bool = True   # R5
    key_property: str = "uri"               # R10
    base_label: str = "Entity"              # R1: one index for all key lookups
    reason_first: bool = False              # R3: OWL 2 RL pre-pass for defined classes
```

Artefact 1: the schema prompt (`owl2lpg.py prompt`)

This is the exact text given to the extraction and text2cypher agents in Chapter 8. It includes hierarchy, synonyms, definitions, cardinality and the transitive/symmetric notes:

```

NODE LABELS (label {properties}) [synonyms]:
  (:AssembledProduct {}) is-a Product
  (:Company {}) is-a Organization
  (:Component {}) is-a Product [aka: part, sub-assembly]
    -- A physical part or sub-assembly that goes into another product. Not the act of assembling.
  (:Country {isoCode: STRING})
  (:Employment {role: STRING}) <n-ary event node>
    -- n-ary: person, employer, role, start, end
  (:Manufacturer {}) is-a Company, Organization
    -- A company that makes finished (assembled) products.
  (:Organization {foundedYear: INTEGER})
  (:Person {})
  (:Product {sku: STRING})
  (:Review {rating: INTEGER})
    -- One customer's written review of one product, with a 1-5 star rating.
  (:Supplier {}) is-a Company, Organization [aka: vendor, provider]
    -- A company that is the supplying party of at least one supply agreement.
  (:SupplyAgreement {unitPriceUSD: FLOAT}) <n-ary event node>
    -- n-ary: supplier, buyer, component, unit price, start date, source
  Properties allowed on any label: endDate: DATE, name: STRING, startDate: DATE
RELATIONSHIPS (from)-[TYPE]->(to):
  (:SupplyAgreement)-[:AGREEMENT_BUYER]->(:Company) // to-one
  (:SupplyAgreement)-[:AGREEMENT_ITEM]->(:Component) // to-one
  (:Company)-[:COMPETITOR_OF]->(:Company) // to-many; symmetric: store once, query undirected
  (:Employment)-[:EMPLOYEE]->(:Person) // to-one
  (:Employment)-[:EMPLOYER]->(:Organization) // to-one
  (:Product)-[:HAS_COMPONENT]->(:Component) // to-many; transitive: store direct links only
  (:Organization)-[:HEADQUARTERED_IN]->(:Country) // to-one; specialises LOCATED_IN
  (:Organization)-[:LOCATED_IN]->(:Country) // to-many
  (:Manufacturer)-[:MANUFACTURES]->(:Product) // to-many
  (:Company)-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]->(:SupplyAgreement) // to-many
  (:Review)-[:REVIEW_OF]->(:Product) // to-one

```

Notice what the compiler did on its own. `manufacturedBy` is gone (R4: its inverse `MANUFACTURES` carries the domain and range). `HEADQUARTERED_IN` inherits its domain and range from `locatedIn` (R5). `name`, `startDate` and `endDate` have no domain, so they are allowed on any label.

Artefact 2: constraints (`owl2lpg.py constraints`)

```

CREATE CONSTRAINT entity_uri IF NOT EXISTS FOR (n:Entity) REQUIRE n.uri IS UNIQUE;
CREATE CONSTRAINT company_uri IF NOT EXISTS FOR (n:Company) REQUIRE n.uri IS UNIQUE;
// ... one per label

```

Artefact 3: instance loading (`owl2lpg.py instances`)

The output is idempotent `MERGE` s, with typed literals (R8), normalised inverse and symmetric edges (R4), a materialised super-property (R5) and no transitive closure (R6):

```

MERGE (n:Entity {uri: ".../acme"}) SET n:Company:Organization,
  n += {foundedYear: 1998, name: "Acme Fasteners GmbH"};
MERGE (n:Entity {uri: ".../empl"}) SET n:Employment,
  n += {endDate: date("2022-06-30"), role: "CT0", startDate: date("2019-01-15")};
MERGE (n:Entity {uri: ".../sa88"}) SET n:SupplyAgreement,
  n += {startDate: date("2021-03-01"), unitPriceUSD: 0.12};
MATCH (a:Entity {uri: ".../acme"}), (b:Entity {uri: ".../DE"}) MERGE (a)-[:HEADQUARTERED_IN]->(b);
MATCH (a:Entity {uri: ".../acme"}), (b:Entity {uri: ".../DE"}) MERGE (a)-[:LOCATED_IN {inferred: true}]->(b);
MATCH (a:Entity {uri: ".../globex"}), (b:Entity {uri: ".../initech"}) MERGE (a)-[:COMPETITOR_OF]->(b);
MATCH (a:Entity {uri: ".../globex"}), (b:Entity {uri: ".../arm200"}) MERGE (a)-[:MANUFACTURES]->(b);

```

(The source data said `initech competitorOf globex`. The symmetric edge was stored in canonical order.)

Artefact 4: validation checks (`owl2lpg.py checks`)

This generates 39 closed-world checks for SupplyKG: endpoint checks (R11), functional checks (R9) and disjointness checks (R14). Each returns violating rows, and an empty result means the check passed. See Chapter 10.

TREAT THE COMPILED ARTEFACTS AS BUILD OUTPUTS Regenerate them in CI whenever the `.ttl` changes, and diff the schema prompt in code review. A one-line ontology change (a new domain, a new functional flag) changes what agents may write. Reviewers should see that.

7.2 The n10s route (Neo4j)

Neosemantics is the reference OWL/RDF ↔ LPG bridge for Neo4j. Its mapping is **generic**: it follows the triples one to one, and it does not create intermediate nodes or refactor properties into nodes for you. Use it to load the **ontology** and **RDF instance data** as they are. Use recipes R1–R14 to decide the model.

```
// required by every n10s import
CREATE CONSTRAINT n10s_unique_uri FOR (r:Resource) REQUIRE r.uri IS UNIQUE;
CALL n10s.graphconfig.init({
  handleVocabUris: 'IGNORE',      // local names (Supplier, hasComponent) instead of ns0__Supplier
  applyNeo4jNaming: true,         // CamelCase labels, UPPER_SNAKE relationship types
  handleMultival: 'ARRAY',        // keep multi-valued literals (OVERWRITE keeps only the last!)
  handleRDFTypes: 'LABELS'       // rdf:type -> label (NODES / LABELS_AND_NODES: option B in R2)
});
CALL n10s.onto.import.fetch("file:///import/supplykg.ttl", "Turtle"); // T-Box -> Class/SC0/... nodes
CALL n10s.rdf.import.fetch("file:///import/instances.ttl", "Turtle");  // A-Box -> nodes and edges
```

What `onto.import` produces:

OWL	n10s LPG
<code>owl:Class</code>	<code>(:Resource:Class {uri, name})</code>
<code>owl:ObjectProperty</code>	<code>(:Resource:Relationship {uri, name})</code>
<code>owl:DatatypeProperty</code>	<code>(:Resource:Property {uri, name})</code>
<code>rdfs:subClassOf</code> / <code>rdfs:subPropertyOf</code>	<code>(:Class)-[:SC0]->(:Class) / -[:SP0]-></code>
<code>rdfs:domain</code> / <code>rdfs:range</code>	<code>-[:DOMAIN]-> / -[:RANGE]-></code>
characteristics (transitive, functional, ...)	a <code>propCharacteristics</code> property (in recent source; not documented)
<code>some</code> / <code>only</code> / cardinality restrictions	<code>-[:SC0_RESTRICTION {onPropertyName, restrictionType}]-></code> (in recent source)
<code>owl:inverseOf</code> , <code>disjointWith</code> , <code>unions</code> , <code>individuals</code>	dropped

Query-time inference over the stored hierarchy (nothing is materialised):

```
CALL n10s.inference.nodesLabelled('Company', {catLabel: 'Class', subCatRel: 'SC0'})
YIELD node RETURN node.name;
MATCH (c:Company {name: 'Acme Fasteners GmbH'})
CALL n10s.inference.getRels(c, 'LOCATED_IN', {subRelRel: 'SP0'})
YIELD rel, node RETURN type(rel), node.isoCode;
```

N10S GOTCHAS (1) The graph config is fixed once any data is loaded. Decide before the first import. (2) Under the default `SHORTEN`, even n10s's own schema names get prefixed (`n4sch__Class`). (3) Blank nodes become `bnode://...` nodes that can't be deleted, and they duplicate on re-import, so skolemise first. (4) Classic `rdf:Statement` reification gets no special handling. RDF-star quoted triples in subject position *do* become relationship properties. (5) The docs and the source disagree on some parameter names and defaults. Pass inference parameters explicitly. (6) n10s is not available on Aura.

7.3 The Neptune route

Amazon Neptune stores everything as quads internally, but it exposes **two separate worlds**: property graph (Gremlin and openCypher) and RDF (SPARQL). One cluster can hold both, but **you cannot query RDF data with openCypher, or the reverse**. For an openCypher KG built from an OWL ontology, the conversion therefore happens **before** loading.

Neptune constraints that shape the mapping:

Neptune fact	Consequence
The only uniqueness guarantee is <code>~id</code>	use the IRI as <code>~id</code> (R10); check natural keys in the loader
No list-valued properties in openCypher; multi-valued properties are read non-deterministically	non-functional datatype properties → value nodes (R12) or delimited strings
No APOC, no n10s, limited <code>CALL</code> procedures	inference is plain Cypher (<code>SC0*0..</code> paths, materialisation jobs)
Bulk loader formats are <code>opencypher</code> CSV (PG) or N-Triples/Turtle/... (RDF); the format decides the world	emit CSV from the RDF view
Neptune Analytics can load N-Triples and expose them to openCypher (IRIs as ids and labels)	good for exploration, but you get raw triple shape, not recipes R2–R7

`owl2lpg.py` `neptune` writes bulk-load CSVs:

```
:ID,:LABEL,endDate:DateTime,foundedYear:Int,isoCode:String,name:String,rating:Int,role:String,sku:String,startDate:DateTime,unitPriceUSD:Double
d:acme,Company;Organization,,1998,,Acme Fasteners GmbH,,,,,
d:emp1,Employment,2022-06-30,,,,,CTO,,2019-01-15,
d:sa88,SupplyAgreement,,,,,,,,,2021-03-01,0.12
...
:ID,:START_ID,:END_ID,:TYPE
e0,d:acme,d:DE,HEADQUARTERED_IN
e7,d:globex,d:arm200,MANUFACTURES
e8,d:initech,d:globex,COMPETITOR_OF
```

(IRIs are shortened to `d:` here for print. The file holds full IRIs.)

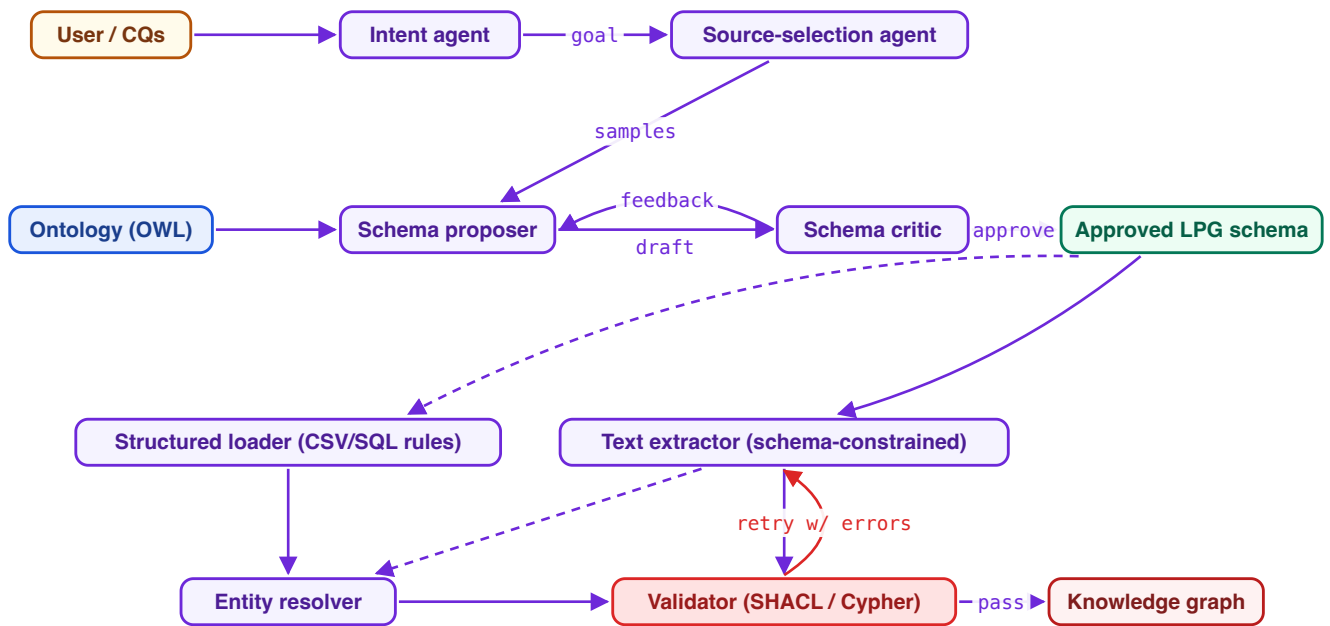
VERIFY AGAINST YOUR ENGINE VERSION Temporal types and functions (`date()` vs `datetime()`) and the `CALL` subquery support differ between Neo4j, Neptune Database and Neptune Analytics, and they change between releases. Keep `owl2lpg.py`'s type map (`NEPTUNE_TYPES`) aligned with the Neptune docs for your version.

7.4 Which route?

Situation	Route
Neo4j, and you want the ontology queryable inside the DB	<code>n10s_onto.import</code> (T-Box) + your own mapping for instances
Neo4j or Neptune, and you need n-ary nodes, typed literals and shortcut edges	<code>owl2lpg</code> -style compiler (recipes R1–R14)
Data already exists as RDF and you just want to explore it with openCypher	Neptune Analytics N-Triples load, or <code>n10s_rdf.import</code>
You need OWL reasoning	reason over the RDF view (Ch. 3), then load the result; never reason inside the LPG

Part III · 8. Ontology-grounded agentic construction

Architecture adapted from the DeepLearning.AI × Neo4j course “Agentic Knowledge Graph Construction”. Its lesson notebooks are publicly mirrored, and the official companion repo is `neo4j-contrib/agentic-kg`. The course derives its schema bottom-up from files and user intent. This chapter adds the ontology as the contract.



Ontology-grounded multi-agent construction. The shape follows the DeepLearning.AI/Neo4j course design (intent → sources → schema loop → construction), extended here with an ontology input, an explicit validator, and a repair loop.

8.1 Design principles from the course

1. **Proposed vs approved state.** Every phase writes a *working* value (`perceived_user_goal` , `proposed_construction_plan`). Only an explicit approve tool promotes it (`approved_*`), and later phases read only `approved_*` keys. This separates the agent’s scratchpad from the specification, which you then persist for reproducibility.
2. **“Trust, but verify” inside tools.** Tools check their preconditions and return errors that tell the model how to recover (“path must be relative”, “column `supplier_id` not found in file”).
3. **A uniform tool envelope:** `{'status': 'success', ...}` or `{'status': 'error', 'error_message': ...}`.
4. **Prompts in four blocks:** role and goal · hints and design rules · output definition · numbered steps that name the tools.
5. **A human in the loop at every phase boundary.** Nothing is approved without the user.
6. **Deterministic where possible.** Loading structured data from an approved plan uses **no LLM**.

State passed between phases:

```

approved_user_goal      {kind_of_graph, graph_description}
approved_files          [paths]
approved_construction_plan {rule_name: node-rule | relationship-rule}      # structured sources
approved_entity_types   [labels]                                          # unstructured sources
approved_fact_types      {predicate: {subject_label, predicate_label, object_label}}
feedback                critic output: 'valid' | 'retry' + bullet list

```

8.2 Where the ontology plugs in

In the course, the schema is *invented* by a proposer agent from file samples. With an ontology, the agents' job changes from **inventing** a schema to **mapping sources onto a fixed one**:

Course step	Without an ontology	With the ontology
well-known types	node labels from the approved plan	<code>owl:Class</code> → labels + <code>rdfs:comment</code> + <code>skos:altLabel</code> (R1, R13)
fact types	proposed by an LLM from text	<code>owl:ObjectProperty</code> domain/range → patterns (R11)
node properties	columns picked by the LLM	<code>owl:DatatypeProperty</code> with XSD types (R8)
unique key	column checked for uniqueness	<code>owl:hasKey</code> / functional key properties (R10)
critic's "redundant inverse" check	heuristic	<code>owl:inverseOf</code> (R4)
critic's "connected schema" check	heuristic	CQs as path tests

The LLM is still useful. It maps *source columns* to ontology properties, proposes **ontology extensions** (new classes it keeps seeing), and critiques. But it can no longer silently change the schema.

8.3 Structured sources: proposer + critic + deterministic loader

Construction-rule format. These are the course's keys, constrained here to ontology terms:

```

{
  "Component": {"construction_type": "node", "source_file": "parts.csv", "label": "Component",
    "unique_column_name": "part_id", "properties": {"part_name": "name", "sku": "sku"}},
  "SupplyAgreement": {"construction_type": "node", "source_file": "part_supplier_mapping.csv",
    "label": "SupplyAgreement", "unique_column_name": "contract_id",
    "properties": {"unit_cost": "unitPriceUSD", "start": "startDate"}},
  "AGREEMENT_ITEM": {"construction_type": "relationship", "source_file": "part_supplier_mapping.csv",
    "relationship_type": "AGREEMENT_ITEM", "from_node_label": "SupplyAgreement",
    "from_node_column": "contract_id", "to_node_label": "Component", "to_node_column":
      "part_id"}
}

```

Note what the ontology changes here. In the course, a supplier-part mapping file becomes a `Supplied_By` {unit_cost, lead_time_days, ...} **relationship**. In SupplyKG, the same file becomes a `SupplyAgreement` **node** (R7), because agreements carry several qualifiers and are CQ answers in their own right.

Proposer tools (course signatures, plus ontology checks):

```

propose_node_construction(approved_file, proposed_label, unique_column_name,
                          proposed_properties, tool_context)
# course: error if unique_column_name not found in file
# + ontology: error if proposed_label not in schema["nodes"],
#             or if a property isn't allowed on the label
propose_relationship_construction(approved_file, proposed_relationship_type,
                                  from_node_label, from_node_column,
                                  to_node_label, to_node_column, proposed_properties, tool_context)
# + ontology: error unless (from_label, type, to_label) is a legal pattern (R11);
#             return the legal patterns as a hint
search_file(file_path, query) # grep, used by both agents to verify uniqueness claims

```

Critic prompt (course checks 1–6, plus ontology checks 7–8; my wording):

You review a proposed mapping from source files into a fixed property-graph schema derived from an OWL ontology. You have read-only tools. For each rule, check: (1) the unique column really is unique (use `search_file`), with no composite keys; (2) no proposed node is really a join table that should be a relationship, *unless* the ontology models it as an n-ary class; (3) you can answer one competency question by tracing sample rows by hand; (4) every label is connected; (5) no hierarchical containment is missing; (6) no relationship duplicates another or is its inverse; (7) every label, type and pattern exists in the schema; (8) every mapped column's values fit the property's datatype. Reply exactly `valid`, or `retry` followed by a bullet list of problems.

Loop (course code, ADK):

```

class CheckStatusAndEscalate(BaseAgent):          # deterministic stop condition
    async def _run_async_impl(self, ctx):
        stop = ctx.session.state.get("feedback", "valid") == "valid"
        yield Event(author=self.name, actions=EventActions(escalate=stop))

schema_refinement_loop = LoopAgent(name="schema_refinement_loop", max_iterations=2,
    sub_agents=[schema_proposal_agent, schema_critic_agent, CheckStatusAndEscalate(name="StopChecker")])

```

Deterministic loader (course, Cypher 5 dynamic labels). All node rules run first, then all relationship rules:

```

LOAD CSV WITH HEADERS FROM 'file:/// ' + $source_file AS row
CALL (row) {
    MERGE (n:{$label}) {uri: $iri_prefix + row[$unique_column_name]}
    FOREACH (k IN keys($properties) | SET n[$properties[k]] = row[k])
} IN TRANSACTIONS OF 1000 ROWS

```

On Neptune there is no `LOAD CSV`, so generate bulk-load CSVs from the plan instead (Ch. 7.3).

8.4 Unstructured sources: schema-constrained extraction

The course's two-step unstructured phase (an **NER agent** proposes entity types, then a **fact-type agent** proposes triples that “must actually appear in the text”) collapses into one step when you have an ontology. The types and patterns come from the compiler. The agents only propose **extensions**, which go to a human.

Extraction prompt (my wording, built around the compiled schema prompt):

You extract facts for a supply-chain knowledge graph.

```
<schema>
{output of owl2lpg.py prompt}
</schema>
```

```
<context>
Document: {title} Source: {uri} Date: {source_date}
{first lines of the document: who/what the document is about}
</context>
```

Rules:

1. Use ONLY the node labels, properties and (from)-[TYPE]->(to) patterns in <schema>. Pick the most specific label that is certainly true (Component, not Product).
Synonyms in [aka: ...] map to that label.
2. Facts with qualifiers (price, dates, role) are event nodes (<n-ary event node>), never properties on an edge.
3. Values must match the declared type: FLOAT is a number, DATE is YYYY-MM-DD. If only a year is known, use YYYY-01-01 and add "datePrecision": "year".
4. Do not infer facts that the text does not state. Store transitive relations as direct links only.
5. For every node and relationship, include "evidence": the shortest verbatim span that supports it.
6. If an important fact does not fit the schema, put it in "unmapped" with a proposed label or type.
Do not force it into the schema.

```
Return JSON: {"nodes": [{id, label, properties, evidence}],
              "relationships": [{start, type, end, properties, evidence}],
              "unmapped": [...]}
```

THE CONTEXT HEADER TRICK The course injects the first lines of each file (the product title and source) into every chunk's prompt inside <context> tags. A chunk that says "five stars, the gearbox died" is then extracted knowing *which product* it is about. This is cheap and very effective.

LIBRARY EQUIVALENTS neo4j-graphrag's SimpleKGPipeline(schema={"node_types", "relationship_types", "patterns", "additional_node_types": False, "additional_patterns": False}) and LangChain's LLMGraphTransformer(allowed_nodes=..., allowed_relationships=[(src, TYPE, tgt), ...], strict_mode=True) both accept the compiled schema directly. Both *filter* output that is off-schema. The guard below also *explains* the problem, which the repair turn needs.

8.5 The schema guard and repair loop

Filtering throws information away. A guard that returns **actionable errors** lets the extractor fix its own output in one more turn. `examples/schema_guard.py` validates extraction JSON against the compiled schema.

Input text: "ACME Fasteners has supplied the X-7 bolt to Globex since 2021 at 12 cents a piece. Their CEO, Dana Kim, says the RoboArm 200 now ships with a new gearbox."

Output: the guard's errors for a plausible first LLM attempt.

```
{
  "node": "n4",
  "error": "unitPriceUSD must be FLOAT, got '12 cents'"
}
{
  "node": "n4",
  "error": "startDate must be an ISO date YYYY-MM-DD, got '2021'",
  "hint": "if only the year is known, use YYYY-01-01 and set datePrecision: 'year'"
}
{
  "node": "n5",
  "error": "unknown label CEO",
  "hint": "use one of ['AssembledProduct', 'Company', ..., 'Person', ...]"
}
{
  "node": "n6",
  "error": "property color not allowed on AssembledProduct",
  "hint": "allowed: ['endDate', 'name', 'sku', 'startDate']"
}
{
  "rel": "(Supplier)-[SUPPLIES]->(Component)",
  "error": "pattern not in ontology",
  "hint": "no relationship type SUPPLIES"
}
{
  "rel": "(Component)-[MANUFACTURES]->(AssembledProduct)",
  "error": "pattern not in ontology",
  "hint": "legal uses of MANUFACTURES: ['(Manufacturer)-[MANUFACTURES]->(Product)']"
}
```

Every error is one the LLM can fix. CEO becomes a Person plus an Employment {role: 'CEO'}. The invented SUPPLIES edge is dropped, because the agreement node already says it. The reversed MANUFACTURES gets flipped. The subclass-aware pattern check means (Supplier)-[PARTY_TO_AGREEMENT_AS_SUPPLIER]->(…) passes even though the pattern is declared on Company.

```
def extract_with_repair(chunk, context, schema, schema_text, max_turns=2):
    msgs = [extraction_prompt(schema_text, context, chunk)]
    for turn in range(max_turns + 1):
        out = parse_json(call_llm(msgs)) # provider-agnostic
        errors = schema_guard.check(out, schema)
        if not errors:
            return out, turn
        msgs += [assistant(out), user("Fix these schema violations and return the full JSON again:\n"
                                      + "\n".join(json.dumps(e) for e in errors))]
    return quarantine(out, errors) # never write unvalidated output
```

WHERE EACH CHECK LIVES Guard (pre-write, per chunk): labels, properties, types, patterns. **Batch validation (pre-commit)**: cardinality, required edges, disjointness, cross-chunk consistency (Ch. 10). **Post-load monitoring**: drift and quality metrics (Ch. 11). Each layer catches what the one before it cannot see.

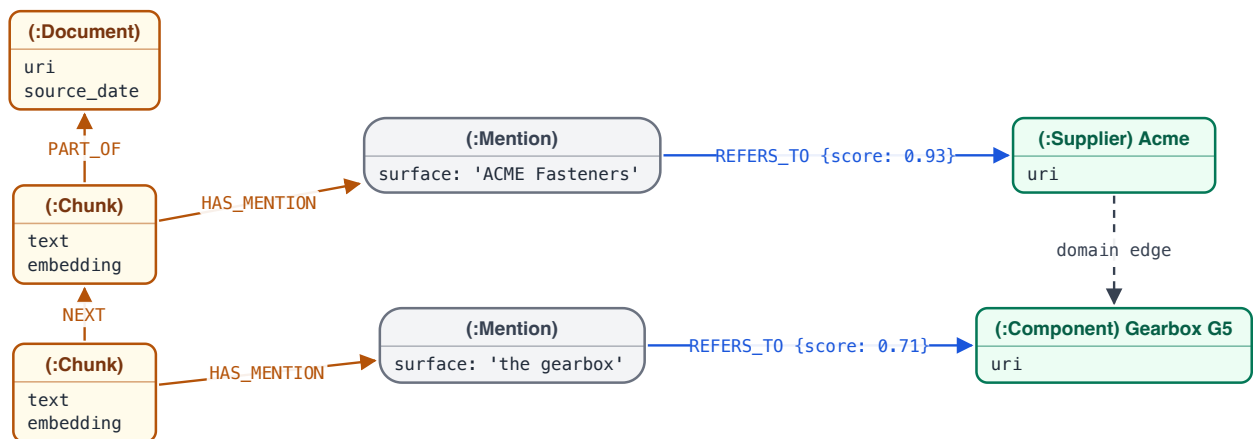
8.6 Provenance, confidence and claims

Every extracted element should answer the questions *where from, when, by which run, how sure*:

```
MERGE (x:Extraction {id: $run_id}) SET x.model = $model, x.prompt_hash = $h, x.at = datetime()
MERGE (d:Document {uri: $doc_uri}) SET d.source_date = date($source_date)
// for every node / relationship written in this run:
// Neo4j (on Neptune, use EXTRACTED_BY edges instead of a list):
SET n.extraction_ids = coalesce(n.extraction_ids, []) + $run_id
// nodes only; for edges, see below
MERGE (n)-[:EVIDENCED_BY {span: $evidence}]->(c:Chunk {uri: $chunk_uri})
```

- For **edges**, provenance is a property (source, evidence, run_id), since an edge can't point at a chunk. Facts that need real evidence links should be **event nodes** (R7).
- Claims vs facts**. Write low-confidence or conflicting facts as (:Claim {type, confidence}) nodes linked to their subject, object and evidence. Promote a claim to a domain edge only after validation, a second source, or human review. Retraction then means deleting a claim, not surgery on the domain graph.

8.7 The lexical graph and linking



The lexical graph (documents, chunks, mentions) linked to the domain graph by entity resolution. Keeping `:Mention` separate keeps the evidence and lets a wrong link be undone without touching the domain graph.

The course builds three layers: a **domain graph** (from CSVs), a **lexical graph** (Document, Chunk with embeddings), and a **subject graph** (LLM-extracted entities, labelled `__Entity__`). It links subject $\rightarrow$ domain with a `CORRESPONDS_TO` edge, using fuzzy key matching and Jaro–Winkler on values. With an ontology, use the same pattern with explicit **mentions**:

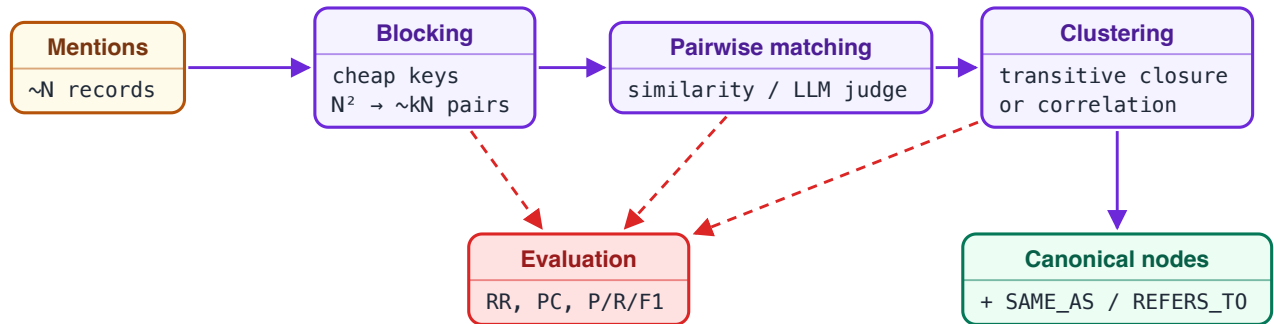
- `(:Chunk)-[:HAS_MENTION]->(:Mention {surface, label})-[:REFERS_TO {score, method}]->(:Supplier)`.
- A wrong link is then one deleted edge. The domain graph is untouched, and the evidence survives.
- **This is the bridge GraphRAG retrieval needs.** PPR seeds come from query $\rightarrow$ mention $\rightarrow$ entity. Answers cite chunks via `EVIDENCED_BY`.

ALL-PAIRS LINKING The course's linking query compares every extracted node with every domain node of the same label, which is fine for a demo and a cartesian product at scale. Add **blocking** (Ch. 9) or a vector-index top-k first.

9. Entity resolution

From Kejriwal's CS520 talk and papers, the CS520 week-4 notes (Chaudhri), and standard ER practice.

Extraction gives you **mentions**. The KG needs **entities**. Because OWL has no unique-name assumption and an LPG has physical node identity, entity resolution (ER) is the step that decides which mentions are the same thing. Kejriwal's talk frames web-scale ER around four requirements that must hold together: it must be domain-independent, automated, scalable and able to handle heterogeneous data.



The classic entity-resolution pipeline and where each metric is measured.

9.1 The pipeline

1. **Type and property alignment.** Only compare mentions of compatible types (ontology labels), on aligned properties (`name` ↔ `name` , `sku` ↔ `sku`). The ontology makes this step nearly free.
2. **Blocking.** Use cheap keys to avoid comparing all N^2 pairs, e.g. token-based keys on a normalised name, `SameFirstDigit(zip)` , or a vector-index top-k. Kejriwal's learned *DNF blocking schemes* combine these, e.g. `(CommonToken, name) v (SameCountry, hq)` .
3. **Pairwise matching.** Score each candidate pair with string similarity (Jaccard, Jaro–Winkler, TF-IDF cosine), embedding cosine, attribute agreement, or an **LLM judge**.
4. **Clustering.** Turn pairwise matches into entities. Connected components are simple but chain errors ($a \sim b, b \sim c \Rightarrow a \sim c$). Correlation or centre clustering is more robust.
5. **Canonicalise and link.** Create or choose the canonical node, then write `REFERS_TO` / `SAME_AS` (Ch. 4.2.2).

9.2 Metrics

Let Ω be all candidate pairs, Ω_m the true matches, Γ the pairs that survive blocking, and $\Gamma_m = \Gamma \cap \Omega_m$.

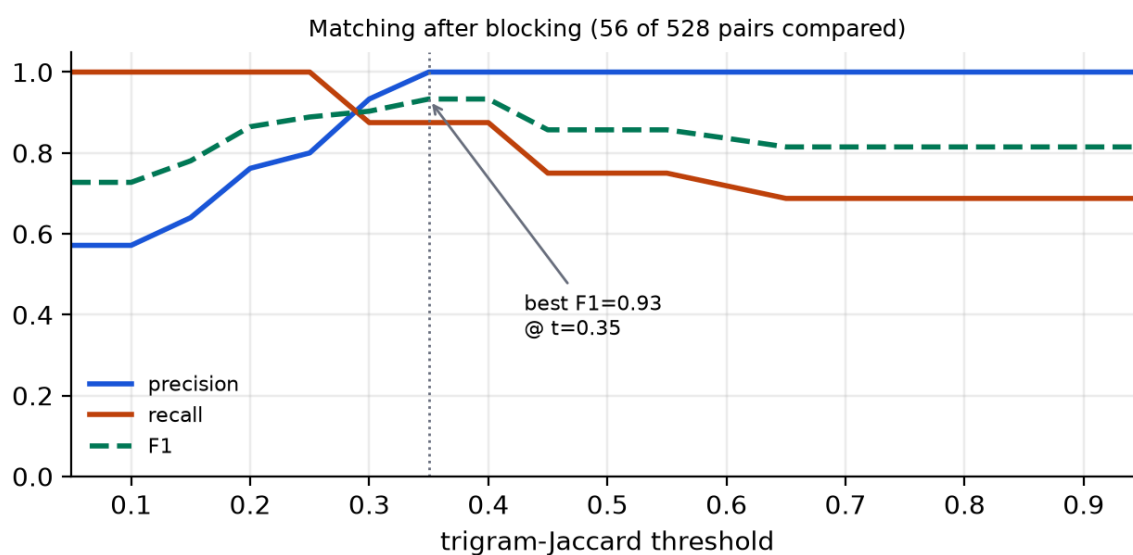
Metric	Formula	Meaning
Reduction ratio	$RR = 1 - \Gamma / \Omega $	how much work blocking saved
Pairs completeness	$PC = \Gamma_m / \Omega_m $	recall of blocking, an upper bound on final recall
Pairs quality	$PQ = \Gamma_m / \Gamma $	precision of blocking
Blocking F	$2 \cdot RR \cdot PC / (RR + PC)$	harmonic mean used by Kejriwal & Miranker
Matching P / R / F1	on predicted match pairs vs gold	final quality
B ³ precision/recall	per-mention cluster overlap, averaged	cluster-level quality (standard ER practice)

9.3 A worked experiment

`examples/er_demo.py` resolves 33 LLM-extracted company mentions into 15 gold entities. It deliberately includes near-collisions: *Acme Fasteners* vs *Acme Robotics*, *Umbrella* vs *Umbra*, *Wayne Enterprises* vs *Wayne Components*, *Initech* vs *Initrode*. The steps are: normalise (lower-case, drop legal suffixes and parentheticals), block on the first 4 characters of the first token, then match on character-trigram Jaccard.

Output (threshold table excerpted):

```
mentions=33 all pairs=528 gold matches=32
blocking: candidate pairs=56 reduction ratio=0.894 pair completeness=1.000
threshold precision recall F1
0.20 0.76 1.00 0.86
0.25 0.80 1.00 0.89
0.30 0.93 0.88 0.90
0.35 1.00 0.88 0.93 <- best
0.50 1.00 0.75 0.86
false merges at best threshold: []
missed matches at best threshold: [('ACME Fasteners', 'Acme', 0.29),
  ('Acme', 'acme fasteners gmbh (Stuttgart)', 0.29), ('Acme Fasteners G.m.b.H.', 'Acme', 0.29),
  ('Acme Fasteners GmbH', 'Acme', 0.29)]
```



Output of `examples/er_demo.py` on 33 toy company mentions. Precision climbs fast; the recall that never comes back is the bare mention "Acme", which is ambiguous between two companies and needs context (attributes or an LLM judge), not a better string metric.

Reading the result:

- Blocking removed **89%** of the comparisons and lost **no** true matches (PC = 1.0). Always report PC. If it drops, no matcher can recover.
- Every remaining miss involves the bare mention “Acme”. It is *genuinely ambiguous* between Acme Fasteners and Acme Robotics, and no string threshold can fix that. It needs **context**: the document’s other entities, attributes such as country or SKU, or an LLM judge that sees both.

9.4 The LLM judge (for the ambiguous band)

Use cheap similarity for the clear cases ($\geq$ upper threshold $\rightarrow$ match, $\leq$ lower threshold $\rightarrow$ non-match), and send only the ambiguous band to an LLM, with **attributes and neighbourhood**, not just names:

```
Decide whether mention M refers to candidate entity E.
Answer JSON {"match": true|false|"unsure", "reason": ""}.
M: surface="Acme" label=Supplier document="QA report RA-200 gearbox failures, 2026-08"
  co-mentioned: ["Gearbox G5", "Globex Robotics", "Stuttgart plant"]
E1: Acme Fasteners GmbH (Supplier) hq=DE supplies=[Bolt X-7] sku_prefix=BLT
E2: Acme Robotics Inc. (Manufacturer) hq=US makes=[ArmBot]
Rules: prefer "unsure" over guessing.
      Matching names alone are not evidence when several candidates share them.
```

MAKE “UNSURE” A FIRST-CLASS OUTCOME Unsure mentions stay as (:Mention) nodes with no REFERS_T0, or with several low-score candidate edges. A wrong merge is far more expensive than a missing link: it corrupts every downstream traversal and every PPR walk.

9.5 ER in Cypher

```
// Candidate generation with a blocking key; embedding / vector top-k works the same way
MATCH (m:Mention {label: 'Supplier'}) WHERE NOT EXISTS { (m)-[:REFERS_T0]->() }
MATCH (e:Supplier) WHERE e.blockKey = m.blockKey
WITH m, e, apoc.text.jarowinklerDistance(toLower(m.surface), toLower(e.name)) AS d // Neo4j + APOC
WHERE d < 0.15
MERGE (m)-[:REFERS_T0]->(e) SET r.score = 1 - d, r.method = 'jw', r.at = datetime()
```

On Neptune (no APOC), compute the similarities in the pipeline and write only the resulting REFERS_T0 edges.

9.6 Schema mapping for structured sources

The CS520 week-4 notes separate **schema mapping** (source tables and columns $\rightarrow$ KG classes and properties) from **record linkage** (is this row an existing entity?). Schema mapping resists automation: opaque column names, business logic, and little training data. So bootstrap it and verify:

- **Linguistic**: equal or stemmed names, synonyms. This is where `skos:altLabel` pays off.
- **Instance-based**: inferred datatypes (a date column can only map to a date property).
- **Constraint-based**: a unique numeric column is probably a key.

This is exactly the proposer/critic loop of Chapter 8.3, and the critic’s uniqueness and type checks are these heuristics.

10. Validation: OWL vs SHACL vs Cypher checks

Chapter 3 argued that OWL is the wrong tool for catching extraction errors. This chapter proves it on one deliberately broken batch, `examples/llm_batch_bad.ttl`, and then shows the same checks running natively on the LPG.

```
# A batch an LLM extractor produced from review text and a contract PDF. Every fact has a defect.
d:rev9  a :Review ; :reviewOf d:acme ; :rating 7 .           # review of a company; rating out of range
d:sa91  a :SupplyAgreement ; :agreementItem d:arm200 ;      # item is an AssembledProduct; no
buyer/supplier
:unitPriceUSD "twelve cents" ; :startDate "2023-01-01"^^xsd:date .
d:emp7  a :Employment ; :employee d:alice ; :employer d:initech ;
:startDate "2024-05-01"^^xsd:date ;
:endDate "2023-02-01"^^xsd:date .                          # ends before it starts
d:umbrella a :Company ; :name "Umbrella Corp" , "Umbrella Corporation" ; # two names
:headquarteredIn d:US , d:DE .                               # functional property, two values
```

10.1 Three validators, one batch

`examples/validate_demo.py` runs OWL 2 RL reasoning, then SHACL with RDFS pre-inference, then SHACL without it. Results from the real run:

#	Defect	OWL 2 RL	SHACL (rdfs inference first)	SHACL (no inference)
1	review of a company	✓ inconsistency (via disjointness)	✗ hidden: range retyped acme as Product	✓ Value does not have class :Product
2	rating 7 > 5	✗	✓	✓
3	agreement item is an assembled product	✓ inconsistency (Component ⊥ AssembledProduct)	✗ hidden	✓
4	no buyer	✗ (open world: a buyer may exist)	✓	✓
5	no supplier	✗ (open world)	✓	✓
6	price is a string	✗	✓ (two violations)	✓ (two violations)
7	employment ends before it starts	✗	✓ (SPARQL constraint)	✓
8	two HQs (functional)	✗ infers US = DE	✓	✓
9	two names (functional)	✗ infers the literals equal	✓	✓
	caught	2 of 9 (+ 1 harmful merge)	7 of 9	9 of 9

SHACL output, no inference:

```
## SHACL view (inference=none): conforms = False
d:emp7      (sparql)      Employment ends before it starts
d:rev9      :rating      Value is not <= Literal("5", datatype=xsd:integer)
d:rev9      :reviewOf    Value does not have class :Product
d:sa91      :agreementBuyer Less than 1 values on d:sa91->:agreementBuyer
d:sa91      :agreementItem Value does not have class :Component
d:sa91      :unitPriceUSD Value is not > Literal("0", datatype=xsd:integer)
d:sa91      :unitPriceUSD Value is not Literal with datatype xsd:decimal
d:sa91      ^:partyToAgreementAsSupplier Agreement has no supplier
d:umbrella  :headquarteredIn More than 1 values on d:umbrella->:headquarteredIn
d:umbrella  :name         More than 1 values on d:umbrella->:name
```

WHY NO INFERENCE IS NEEDED FOR CLASS CHECKS `sh:class` already follows `rdfs:subClassOf` in the data graph. So `d:globex a :Manufacturer` satisfies `sh:class :Company` with no reasoner, as long as the ontology triples are loaded next to the data. Pre-inference only adds range-based retyping, which is exactly what hides errors.

Hogan's summary (§7.3), confirmed here: *consistency* (OWL) and *validity* (shapes) catch **different** problems. Shapes catch missing, extra and malformed data. OWL catches logical contradictions, but only where a negative axiom (disjointness, `differentFrom`, irreflexivity) exists to contradict.

10.2 The same checks on the LPG

SHACL runs over RDF. On the LPG, run the equivalent Cypher, generated from the ontology (`owl2lpg.py` checks), plus a few hand-written business rules:

```
// R11 endpoints: REVIEW_OF must go (Review)->(Product)
MATCH (x)-[r:REVIEW_OF]->(y) WHERE NOT (x:Review AND y:Product) RETURN 'REVIEW_OF' AS check, x.uri, y.uri;
// R9 functional: at most one HQ
MATCH (x)-[r:HEADQUARTERED_IN]->() WITH x, count(r) AS n WHERE n > 1 RETURN 'HQ functional', x.uri, n;
// R14 disjointness
MATCH (n:Component:AssembledProduct) RETURN 'disjoint Component/AssembledProduct', n.uri;
// required participants of an event node (SHACL minCount 1)
MATCH (sa:SupplyAgreement) WHERE NOT EXISTS { (sa)-[:AGREEMENT_BUYER]->() }
OR NOT EXISTS { (:Company)-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]->(sa) }
RETURN 'agreement participants', sa.uri;
// value ranges and datatypes
MATCH (r:Review) WHERE NOT (r.rating IN range(1, 5)) RETURN 'rating range', r.uri, r.rating;
MATCH (sa:SupplyAgreement) WHERE sa.unitPriceUSD IS NOT NULL AND NOT (sa.unitPriceUSD > 0)
RETURN 'price', sa.uri, sa.unitPriceUSD;
// temporal sanity
MATCH (e:Employment) WHERE e.endDate < e.startDate RETURN 'employment dates', e.uri;
```

Run them against a **staging** copy (or a per-batch subgraph tagged by `run_id`) before promoting a batch.

10.3 A validation pipeline that works

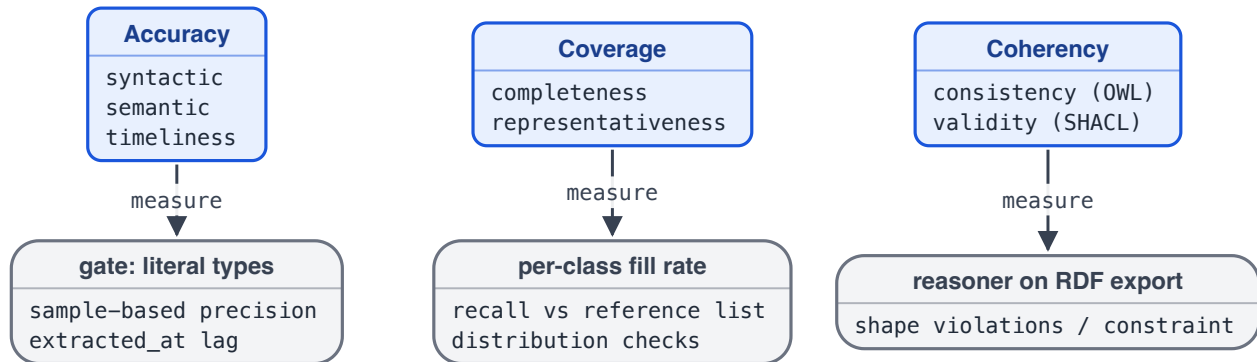
WHEN	WHAT	CHECKS	ON FAILURE
per chunk quarantine	<code>schema_guard.check()</code>	labels, properties, types, patterns	repair turn /
per batch batch	SHACL on the batch's RDF, or Cypher checks on staging	cardinality, required edges, ranges, rules	reject or fix
periodically	OWL reasoner on RDF export	disjointness, differentFrom, defined classes	investigate
periodically	emergent-schema diff	unexpected (labels, type, labels) patterns	drift alert (Ch. 11)

RULES OF THUMB (1) Never run a reasoner over *unvalidated* LLM output. One bad functional fact merges countries. (2) Validate **before** any inference. (3) Keep the OWL reasoner for the jobs only it does well: classifying into defined classes (R3), finding contradictions, and explaining them.

Part IV · 11. Quality and drift monitoring

Distilled from Hogan et al. §7.1–7.3, and made operational in openCypher.

Quality means **fitness for purpose**. Hogan groups it into accuracy, coverage and coherency:



Quality dimensions from Hogan et al. Ch. 7 and how each is measured in practice.

11.1 Accuracy

Syntactic accuracy asks whether values are well-formed for their declared type. The metric is $\text{invalid} \div \text{total}$, per property.

```
// share of SupplyAgreements whose price is not a number (Neo4j 5 type predicates)
MATCH (sa:SupplyAgreement) WHERE sa.unitPriceUSD IS NOT NULL
RETURN 1.0 * sum(CASE WHEN sa.unitPriceUSD IS :: FLOAT OR sa.unitPriceUSD IS :: INTEGER
                     THEN 0 ELSE 1 END) / count(*) AS invalidRatio
```

Semantic accuracy asks whether the values are *true*. No query over the graph can find a swapped supplier. Hogan lists three approaches: manual verification, checking against independent sources, and judging the **extraction process** rather than individual facts. For LLM pipelines that means:

- a gold set of ~200 chunks with hand-labelled facts, where you measure precision and recall per relationship type;
- re-running the gold set whenever the model, prompt or ontology changes (the prompt hash lives on `(:Extraction)`);
- sampling *k* facts per run for human review, stratified by relationship type and confidence.

Timeliness asks whether the graph keeps up with the world. Store stable facts (`foundedYear` , not `age`) and absolute dates (never “next Tuesday”). Track `source_date` and `extracted_at` , and alert on lag.

11.2 Coverage

Aspect	Question	Cypher
Schema completeness	which ontology classes have no instances?	compare <code>owl2lpg.py json</code> labels with <code>CALL db.labels()</code> (or <code>MATCH (n) UNWIND labels(n) AS l RETURN DISTINCT l</code>)
Property completeness	what share of Components lack a SKU?	see below
Population completeness	what share of known suppliers are in the KG?	recall against a reference list (ERP vendor master)
Linkability	what share of entities have an external ID or a <code>REFERS_TO</code> from a mention?	see below

```
// property completeness per label/property, driven by the compiled schema
UNWIND $schema AS s                                // [{label:'Component', prop:'sku'}, ...]
CALL (s) { MATCH (n) WHERE s.label IN labels(n)
            RETURN count(n) AS total, count(n[s.prop]) AS filled }
RETURN s.label, s.prop, filled, total, round(1.0 * filled / total, 3) AS fillRate
ORDER BY fillRate
```

(On Neptune, generate one query per label and property instead of using `CALL` subqueries.)

Representativeness asks whether the sample is biased. Compare distributions with reality: suppliers per country against spend per country, reviews per product against sales. LLM extraction inherits the biases of its sources, and of the model.

11.3 Coherency

- **Consistency:** count OWL inconsistencies on the RDF export, broken down by the axiom that caused them (disjointness, `differentFrom`).
- **Validity:** count shape or check violations, per check. The output of `owl2lpg.py checks` becomes a dashboard.

```
// one validity report from the generated checks (Neo4j).
// On Neptune, run each check separately and aggregate client-side.
CALL () {
  MATCH (x)-[r:REVIEW_OF]->(y) WHERE NOT (x:Review AND y:Product) RETURN 'REVIEW_OF endpoints' AS check
  UNION ALL
  MATCH (x)-[r:HEADQUARTERED_IN]->() WITH x, count(r) AS n WHERE n > 1 RETURN 'HQ functional' AS check
  UNION ALL
  MATCH (n:Organization:Product) RETURN 'disjoint Organization/Product' AS check
}
RETURN check, count(*) AS violations ORDER BY violations DESC
```

11.4 Drift: emergent schema vs ontology

Run the label-set quotient from Chapter 4.1.3 nightly, and diff it against the compiled patterns:

```

observed = run("""MATCH (a)-[r]->(b)
                RETURN [l IN labels(a) WHERE l <> 'Entity'] AS f, type(r) AS t,
                        [l IN labels(b) WHERE l <> 'Entity'] AS g, count(*) AS n""")
legal = {(r["from"], r["type"], r["to"]) for r in schema["relationships"]}
for row in observed:
    if not any((a, row.t, b) in legal for a in row.f for b in row.g):
        alert(f"drift: {row.f}-[{row.t}]->{row.g} x{row.n}")

```

Typical findings: a new relationship type slipped past the guard (someone disabled it), a label that is never used (an extraction gap), or a pattern whose count jumped after a prompt change.

11.5 A minimal quality dashboard

Panel	Source	Alert when
guard rejection rate per run	schema_guard logs	> 10% or a sudden change
repair success rate	extraction loop	< 80%
validity violations per check	generated checks	any increase
fill rate per (label, property)	§11.2	a drop > 5 points
ER: PC, and precision on a sample	Ch. 9	PC < 0.98
drift patterns	§11.4	any new pattern
CQ acceptance tests	Ch. 12.1	any failure

12. From graph to answers, and lessons from industry

12.1 Competency questions as acceptance tests

Every CQ from the front matter becomes a parameterised query with an expected answer on a fixture graph (the instance file). Run them in CI after every ontology or pipeline change:

```
CQ_TESTS = {
  "CQ1": ("MATCH (:AssembledProduct {name:$p})-[:HAS_COMPONENT*1..]->(c:Component)"
    "<-[:AGREEMENT_ITEM]-(:SupplyAgreement)<-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]-(:s) "
    "RETURN collect(DISTINCT s.name) AS r", {"p": "RoboArm 200"}, ["Acme Fasteners GmbH"]),
  "CQ3": ("MATCH (:Country {isoCode:$c})<-[:LOCATED_IN]-(:Company)-[:PARTY_TO_AGREEMENT_AS_SUPPLIER]->"
    "(:SupplyAgreement)-[:AGREEMENT_ITEM]->(:Component)<-[:HAS_COMPONENT*1..]-(:p:AssembledProduct) "
    "RETURN collect(DISTINCT p.name) AS r", {"c": "DE"}, ["RoboArm 200"]),
  "CQ6": ("MATCH (:Company {name:$c})-[:COMPETITOR_OF]-(x) RETURN collect(x.name) AS r",
    {"c": "Globex Robotics"}, ["Initech"]),
}
```

CQ3 only works because `LOCATED_IN` was materialised from `HEADQUARTERED_IN` (R5), and CQ1 only because the transitive closure was *not* stored (R6). Both mapping decisions are now tested.

12.2 text2cypher with the compiled schema

Give the query agent the **same schema text** the extractor used, plus the notes that only a query writer needs:

```
You translate questions into openCypher for the graph described in <schema>.
<schema>{owl2lpg.py prompt}</schema>
Query-writing notes derived from the ontology:
- "manufactured by X" = (X)-[:MANUFACTURES]->(p). No MANUFACTURED_BY type exists (inverse, R4).
- COMPETITOR_OF is symmetric: always match it undirected.
- HAS_COMPONENT is transitive: use [:HAS_COMPONENT*1..] for "contains / uses at any depth".
- LOCATED_IN includes HQ locations. Use HEADQUARTERED_IN only for "headquartered".
- Prices, dates and roles live on :SupplyAgreement and :Employment nodes, not on edges.
- Match names case-insensitively via a Mention/REFERS_TO lookup, or toLower(); never guess IRIs.
Return only the query.
```

ANSWERING UNDER AN OPEN WORLD An empty result means “*not found in the graph*”, not “*no*”, unless the queried region is marked complete (Ch. 3.6). Tell the answering agent which sources are complete, and have it phrase answers accordingly: “The graph lists no other suppliers for Bolt X-7. Contract coverage is complete only for 2024 onward.”

12.3 Hybrid retrieval over an ontology-typed graph

The ontology makes graph retrieval more controllable than an untyped extracted graph:

- **Seeding.** Query entities → `(:Mention)` → `REFERS_TO` → canonical entities, the same path used at indexing time (Ch. 8.7). Mention nodes keep the seeds auditable.

- **Typed walks.** Personalised PageRank can be restricted to (or weighted by) relationship types that are relevant to the question class: BOM questions walk `HAS_COMPONENT` and `AGREEMENT_ITEM`; people questions walk `EMPLOYEE / EMPLOYER`. Event nodes (R7) are natural hubs, since they connect all participants of a fact.
- **Supernode control.** Ontology “value” nodes (`Country`) have huge degree. Down-weight them or exclude them from walks unless the question is about them (Ch. 5.2).
- **Grounded answers.** Every domain fact has `EVIDENCED_BY` chunks or claim evidence (Ch. 8.6), so answers can cite the text span, not just the edge.

12.4 Lessons from the Amazon Product Graph

From Luna Dong’s CS520 talk (“Zero to One Billion: the path for a rich product graph”) and the AutoKnow paper (KDD 2020).

AutoKnow builds a product graph of > 1B triples across > 19K product types in five components. Each one maps onto something in this book:

AutoKnow component	What it does	Equivalent here
Taxonomy enrichment	finds new product types and attaches them to parents (GNN over type–product–query graph)	ontology extension proposals from <code>unmapped</code> (Ch. 8.4); option B hierarchy (R2)
Relation discovery	learns whether attribute A applies to type T, and how important it is	per-class property applicability, a <i>weighted</i> <code>rdfs:domain</code> (R11)
Data imputation	fills missing (attribute, value) pairs from text, conditioned on the type	schema-constrained extraction (Ch. 8.4)
Data cleaning	a transformer scores whether (product, attribute, value) is consistent	learned validation, the statistical cousin of SHACL (Ch. 10)
Synonym discovery	“lite” ≈ “low sugar”, “FTO” ≈ “fair trade organic”	value canonicalisation, <code>skos:altLabel</code> , ER on values (Ch. 9)

Lessons that transfer directly:

1. **Taxonomies are not trees.** Products need multiple parents and overlapping types. Model subtype, synonym and overlap explicitly (R2, R13).
2. **Clean before you learn, and clean continuously.** Their anomaly detector found 1.77M wrong flavour and scent values at 90% precision (“Pink” as a flavour). Expect the same class of error from LLM extraction.
3. **Text is not enough.** About 71% of values came from product profiles, 4% from images and 25% from external sources. Plan for structured and multimodal inputs alongside text.
4. **Dong’s scorecard.** Entity linkage, closed IE, knowledge cleaning and knowledge fusion have succeeded in industry. Automated schema mapping, open IE and knowledge inference largely have not. That is the argument for a fixed ontology plus constrained extraction, rather than open extraction.

Appendix A · End-to-end checklist

Design (Ch. 2–4)

- ☐ Scope, users and 5–15 competency questions are written down, each as a Cypher query.
- ☐ Classes have `rdfs:comment` definitions and `skos:altLabel` synonyms.
- ☐ Disjointness is declared among top-level classes and among sibling subclasses where it holds.
- ☐ Every object property has a domain and range. Functional and key properties are marked.
- ☐ Qualified facts are n-ary classes, decided with the rule in 4.3.

Mapping (Ch. 5–7)

- ☐ `MappingConfig` decisions are recorded: n-ary classes, hierarchy option per branch (R2), key property, materialisation of superproperties and defined classes.
- ☐ The schema prompt, constraints, checks and loader are generated in CI from the `.ttl`.
- ☐ On Neptune, IRIs are `~id`. There are no list properties, and inference runs as plain Cypher.

Population (Ch. 8–9)

- ☐ Agents read `approved_*` state only, and a human approves each phase.
- ☐ Extraction uses the schema prompt plus a per-document context header, and returns evidence spans.
- ☐ Every chunk passes the schema guard (with at most 2 repair turns) or goes to quarantine.
- ☐ Mentions are linked through `REFERS_TO {score, method}`. Merges happen only on keys or with high confidence.
- ☐ ER reports RR and PC for blocking, and P/R on a labelled sample.
- ☐ Every write records its `run_id`, model, prompt hash, source document and date.

Operation (Ch. 10–12)

- ☐ Batches are validated before inference, never after.
- ☐ Generated checks, fill rates and drift run nightly, and the dashboard from 11.5 alerts.
- ☐ The CQ acceptance tests pass.
- ☐ The QA agent knows which regions of the graph are complete, and phrases empty results as “not found”.

Appendix B · Corrections to the sources

These are places where a source is wrong, ambiguous or out of date. The research for this book found them, so you don't have to.

Source	Issue	Correct reading
Ontology 101 §4.1	subclass definition stated backwards	if $A \sqsubseteq B$, every instance of A is an instance of B
Pizza tutorial, anonymous-class example	calls a union ("A or B") an intersection	or is union; listing two ranges gives the intersection
Pizza tutorial	"a transitive property cannot be functional"	OWL 2 DL <i>forbids</i> it syntactically, to keep reasoning decidable
Pizza tutorial §4.13	names Mozzarella/Tomato where Cheese/Vegetable is meant	the point (two only s intersect) stands
Hogan book repo, Fig. 2.2 Cypher	<code>-[borders]-></code> has no colon, so <code>borders</code> is a variable, not a type	<code>-[:borders]-></code>
Hogan book repo, Fig. 7.2	<code>:domain</code> written instead of <code>rdfs:domain</code>	<code>rdfs:domain</code>
Hogan book, Fig. 3.3 ShEx	<code>:name xsd:string *</code> allows zero names, but the text requires ≥ 1	<code>+</code>
GraphAcademy	<code>WHERE exists((p)-[:ACTED_IN]->())</code>	deprecated in Neo4j 5; use <code>EXISTS { ... }</code>
GraphAcademy	dates stored and compared as strings	use <code>date()</code>
n10s docs	reference page parameter names (<code>className</code> , ...) and inference defaults differ from the source	source uses <code>className</code> , <code>subClassOfRel</code> , ...; pass parameters explicitly
n10s docs	constraint syntax <code>ON ... ASSERT</code>	Neo4j 5: <code>FOR (r:Resource) REQUIRE r.uri IS UNIQUE</code>
neo4j-graphrag docs	lexical-graph relationship called both <code>FROM_DOCUMENT</code> and <code>PART_OF_DOCUMENT</code>	check your installed version

Appendix C • References

Primary sources used in this book

- Noy, N. & McGuinness, D. *Ontology Development 101: A Guide to Creating Your First Ontology*. https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html
- DeBellis, M. *A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins* (Pizza Tutorial v3.2). <https://www.michaeldebellis.com/post/new-protege-pizza-tutorial>
- Hogan, A. et al. *Knowledge Graphs*. Synthesis Lectures / ACM CSUR. <https://kgbook.org/> · examples: <https://github.com/Knowledge-Graphs-Book/examples>
- Neo4j GraphAcademy. *Graph Data Modeling Fundamentals*. <https://graphacademy.neo4j.com/courses/modeling-fundamentals/> · source: <https://github.com/neo4j-graphacademy/courses>
- Neo4j Labs. *neosemantics (n10s)*. <https://neo4j.com/labs/neosemantics/> · <https://github.com/neo4j-labs/neosemantics>
- AWS. *Amazon Neptune User Guide*, data model and openCypher compliance. <https://docs.aws.amazon.com/neptune/latest/userguide/feature-overview-data-model.html>
- DeepLearning.AI × Neo4j. *Agentic Knowledge Graph Construction*. <https://www.deeplearning.ai/short-courses/agentic-knowledge-graph-construction/> · companion: <https://github.com/neo4j-contrib/agentic-kg>
- Stanford CS520 *Knowledge Graphs* (Spring 2021). <https://web.stanford.edu/class/cs520/>. Kejriwal slides: <https://web.stanford.edu/class/cs520/abstracts/kejriwal.pdf> · Dong slides: <https://web.stanford.edu/class/cs520/abstracts/dong.pdf>
- Dong, X. L. et al. *AutoKnow: Self-Driving Knowledge Collection for Products of Thousands of Types*. KDD 2020. <https://arxiv.org/abs/2006.13473>
- Kejriwal, M. & Miranker, D. *A DNF Blocking Scheme Learner for Heterogeneous Datasets*. <https://arxiv.org/abs/1501.01694>
- Papadakis, G. et al. *Blocking and Filtering Techniques for Entity Resolution: A Survey*. <https://arxiv.org/abs/1905.06167>
- W3C. *SHACL*. <https://www.w3.org/TR/shacl/>

Further reading: ontology-guided LLM extraction

- Text2KGBench (ontology-driven KG generation benchmark). <https://arxiv.org/abs/2308.02357>
- EDC: Extract, Define, Canonicalize. <https://arxiv.org/abs/2404.03868>
- SPIRES / OntoGPT (schema-driven zero-shot extraction). <https://arxiv.org/abs/2304.02711>
- Ontology-grounded KG construction by LLM under the Wikidata schema. <https://arxiv.org/abs/2412.20942>
- iText2KG (incremental extraction with resolution). <https://arxiv.org/abs/2409.03284>
- KGGen (extraction + entity/relation clustering). <https://arxiv.org/abs/2502.09956>
- Ontology generation using LLMs. <https://arxiv.org/abs/2503.05388> · OLLM: <https://arxiv.org/abs/2410.23584>
- neo4j-graphrag KG builder. https://neo4j.com/docs/neo4j-graphrag-python/current/user_guide_kg_builder.html
- LangChain LLMGraphTransformer (allowed nodes / relationship triples, strict mode).

Appendix D • Files in `examples/`

File	What it is	Chapter
<code>supplykg.ttl</code>	the running-example OWL ontology	all
<code>instances.ttl</code>	clean instance data	3, 7
<code>llm_batch_bad.ttl</code>	a deliberately broken extraction batch	3, 10
<code>shapes.ttl</code>	SHACL shapes derived from the ontology	4, 10
<code>owl2lpg.py</code>	the OWL → LPG compiler: <code>prompt</code> , <code>json</code> , <code>constraints</code> , <code>instances</code> , <code>instances-reasoned</code> , <code>checks</code> , <code>ontology-graph</code> , <code>neptune</code>	6, 7
<code>reason_demo.py</code>	OWL 2 RL inferences and the domain/range trap	3
<code>validate_demo.py</code>	OWL vs SHACL (with and without inference)	3, 10
<code>schema_guard.py</code>	validates LLM extraction JSON with actionable errors	8
<code>er_demo.py</code>	blocking + matching metrics and chart	9

All outputs quoted in this book were produced with `rdflib` 7.6, `owlrl` and `pyshacl` 0.40 in September 2026. The Cypher in this book was written for Neo4j 5 / openCypher. It was not executed against a live database while the book was compiled, so run the CQ tests (12.1) on your own instance first.